arXiv:2609.07065v1 [cs.AI] 7 Sep 2026

VST: Verifiable Structured Transport for Auditable Agent-to-Agent Alpha Discovery

Yuqi Li Siyuan Liu Bingjun Liu

Panda AI
{libubai, liusiyuan, liubingjun}@pandaai.online

Abstract

Agent-to-agent (A2A) alpha discovery is slowed by repeated feedback cycles between mining and evaluation agents, whose hand-offs, in contemporary LLM multi-agent systems, are free-form natural-language messages that carry no stable contract and cannot be replayed. We first restructure this communication as a structured agent-to-agent protocol of *typed, causally addressable, unicast records*, so that the committed stream forms a causal trajectory. On that trajectory a single predictor with four typed heads forecasts the accumulated guidance the two miners would receive several cycles ahead; a transactional verify–leap controller then commits a multi-cycle speculative outcome only when it passes a four-level gate, and otherwise rolls back to the exact prior state. Structure is the enabling contribution, and its value is not accuracy. A controlled ablation shows an equal-information free-text channel reaches the same predictor hit rate. What typing provides is a state that can be schema-checked, replayed deterministically, and prevented by construction from leaking a forecast to an evaluator: auditability by construction, not an empirically stress-tested guarantee. On a CSI 1000 out-of-sample holdout, our single run is the only one among eight methods (seven baselines and ours) to hold a positive median annualized return and Sharpe at the factor level, though the median return *in excess* of the benchmark stays negative for every method including ours; its development-selected top-20 portfolios reach a 0.71 median holdout Sharpe, selected on a split inside the optimization horizon. We report these single-run results descriptively, gross of costs, and are explicit about their limits throughout; in particular we do not isolate the effect of the leap machinery from the inherited search substrate, which we leave to future work.

Keywords: agent-to-agent communication, LLM multi-agent systems, speculative execution, alpha factor mining

1 Introduction

Automated discovery of predictive signals, or *alphas*, has moved from single-model symbolic search toward multi-agent pipelines in which several specialized components propose, evaluate, and refine candidates in concert. A representative design is a dual-loop evolutionary search rather than a linear chain, in which one report agent advises two miners: an alpha-mining loop proposes candidate factors and analyzes them with independent evaluator instances, while in parallel an evaluation-optimization loop proposes changes to the metric set and portfolio construction and analyzes the resulting configuration [1]. The two loops are coupled, because the evaluation side scores alpha candidates while accepted factors become evaluation-side samples. Only after the coupled state converges does the system commit the factor pool, the evaluation configuration, the

report memory, and the approved skills. This dual-loop architecture is the substrate we build on; it is not a contribution of this paper. What we take from it is a concrete, demanding instance of a recurring problem: repeating these miner–evaluator–report cycles is the serial cost we seek to reduce.

The bottleneck lies in the number of round-trips between agents rather than in the computation inside any single agent. Each cycle waits for a miner to produce candidates, for several evaluators to return analyses, and for a report agent to integrate those analyses into both loops. When the search runs for hundreds of cycles, the wall-clock cost is dominated by this sequential feedback rather than by any one model call.

Contemporary LLM multi-agent frameworks show that role specialization and agent-to-agent (A2A) messaging can solve complex tasks [2, 3, 4, 5]. A recurring property of these systems, surveyed in [6], is that agents coordinate primarily through *free-form or loosely structured natural language*. Even where individual hand-offs are typed by structured-output schemas, the interaction as a whole is rarely exposed as a stable, replayable record contract, so the message stream cannot be aligned into a causal trajectory. These properties are acceptable when the only goal is to reach a task answer, but they foreclose two capabilities that matter for a long-running evolutionary search: predicting where the interaction is heading, and verifiably skipping ahead to that state.

Our central idea is that structuring the communication is a prerequisite, not an optimization. We reconstruct every inter-agent transmission as a typed three-layer record, an envelope that identifies and routes the message, a role-specific body that retains the original payload and receiver argument, and a transactional provenance block that records whether the message belongs to a committed or a speculative state (detailed in Section 3.2). The wrapper gives each agent a stable input contract, removes ambiguous prose hand-offs, enforces visibility boundaries, and makes multi-cycle state transitions replayable.

Only once actual communication has this structure do its committed records form a trajectory that can be replayed and aligned into supervised targets. We then introduce one predictor that forecasts the accumulated *miner-directed guidance* the two miners would receive over the next k cycles, and a controller that runs a short speculative execution and commits the result only if it passes a layered validation. A prediction is never acted on directly: it proposes a candidate future state, and the real miners and evaluators then decide whether that state is valid. Structuring the channel is what makes this loop safe and auditable, because the forecast is schema-checked, its routing cannot leak to an evaluator, and a rejected leap rolls back to an exact prior state.

Contributions. Taking the dual-loop agentic alpha-discovery system of [1] as a fixed substrate, this paper turns its communication layer into a structured agent-to-agent protocol: a typed transport on which multi-cycle guidance can be forecast and a speculative leap can be verifiably committed. We call this paradigm VST (Verifiable Structured Transport), where the forecast is a prediction over the structured communication trajectory that is never trusted directly; it proposes a candidate future state that the real agents verify, and we make no claim that structuring improves forecast accuracy. The four contributions below follow from one idea: the inter-agent channel should be a typed, replayable record (C1). Once it is, three capabilities follow that a prose channel cannot support: the trajectory can be forecast (C2), a forecast can be verifiably committed or rolled back (C3), and the predictor can co-evolve without corrupting the evaluators that judge it (C4).

- **C1: Structured inter-agent communication protocol.** A three-layer, A2A-compatible packet format with deterministic unicast routing and visibility typing that preserves the existing payload field paths and receiver arguments while making every transmission typed, causally addressable, and replayable (Section 3.2). This is the enabling contribution. Its

value is not accuracy: a controlled ablation (Section 4.7) shows an equal-information free-text channel reaches the same predictor hit rate. What typing provides is a leak-proof, replayable trajectory at no cost to downstream effectiveness, an architectural correctness property the free-text channel cannot provide and on which the verified leap depends.

- **C2: Multi-horizon structured trajectory prediction.** A single agentic-RL predictor with four typed heads that jointly forecasts, for a chosen horizon $k \geq 2$, the accumulated guidance delivered to the two miners, with a calibrated success confidence and epistemic uncertainty (Section 3.3). Two design choices are specific to our setting rather than inherited from standard LLM-agent RL: a single shared-encoder policy with four typed heads (not four independent agents), which keeps the coupled channels cross-consistent; and a transaction-level, verification-grounded reward, in which the four-level gate's commit/rollback outcome is the learning signal and rolled-back trajectories are retained as negatives (Section 3.5).

- **C3: Transactional verification and leap control.** A four-level gate that validates the speculative *outcome* rather than the forecast field-by-field, committing the complete transaction atomically or rolling it back to the last committed state (Section 3.4).

- **C4: Separated skill co-evolution.** Distinct, transactionally promoted skill libraries for the predictor and the evaluators, which prevent a successful predictor from teaching the evaluators to agree with its own forecast (Section 3.5).

The design borrows the verify-then-commit discipline of speculative decoding [7] but applies it at the level of a multi-agent workflow: the unit of speculation is a multi-cycle transition of two coupled loops, not a token.

2 Related Work

2.1 LLM multi-agent systems and their communication layer

We position our work against several literatures of unequal weight. How LLM agents communicate (this subsection) is where our load-bearing contribution lives; prediction and learning over agent trajectories, automated alpha discovery and financial LLM agents, and speculative execution are the traditions from which our downstream consequences borrow. Role-specialized LLM agents now collaborate on multi-step tasks through a variety of coordination structures: fixed assembly-line workflows with standardized roles [2, 3], general frameworks for composing conversational agents [4, 5], and looser regimes in which agents debate, recompose their team, or simulate social behavior [8, 9, 10]; recent work scales this to networked topologies with a reported collaborative scaling law [11]. What these designs share, and what several surveys now foreground as a first-class design axis [6, 12, 13, 14], is that the coordination medium itself is natural language.

Across these designs the primary coordination medium is natural language. Modern stacks do impose typed contracts on individual hand-offs (structured output, JSON-mode, function/tool-calling), and systems such as MetaGPT exchange structured intermediate artifacts rather than pure prose; what remains absent is not per-message typing but a *transactional, replayable record contract* over the whole interaction. Typed performative envelopes that separate routing metadata from content are themselves long-standing: agent communication languages such as KQML [15] and the FIPA-ACL tradition established them decades ago, and the Model Context Protocol and Agent2Agent protocol revive them today. Our envelope sits in this lineage; what we add to it is the *transactional provenance* layer (observed vs. speculative origin, commit status) and the

visibility typing that together make a speculative multi-cycle transaction replayable, leak-proof, and rollback-safe, properties an ACL envelope alone does not provide. This also departs from multi-agent reinforcement learning, where agents *learn* a differentiable communication protocol end-to-end [16], whereas contemporary LLM agents exchange free-form text. Our work is orthogonal to the coordination pattern: rather than propose a new role structure or prompting scheme, we replace the untyped prose channel with a typed, replayable transport on which the same information becomes a stable trajectory coordinate. Closest to our setting, a dual-loop agent-to-agent architecture for self-evolving alpha discovery has been proposed [1], in which coupled mining and evaluation loops co-evolve through repeated feedback. That work establishes the multi-agent search substrate; it communicates through conventional hand-offs and does not treat the communication layer itself as an object to be typed, forecast, and verifiably advanced, which is the gap this paper addresses.

2.2 Prediction and learning over agent trajectories

Our predictor is an agentic reinforcement-learning component, and it forecasts a future state before acting on it, so it draws on two adjacent lines. The first turns an LLM into a policy: from prompting techniques that elicit or search over reasoning [17, 18], through agents that interleave reasoning with tool use and self-critique [19, 20, 21, 22], to policies trained with PPO-style RLHF and, more recently, group-relative objectives that incentivize reasoning directly [23, 24, 25, 26]. Voyager's growing library of executable skills [27] directly motivates our predictor skill library (Section 3.5). These settings are single-agent and optimize toward an answer or action sequence; our reward is instead a *transaction-level* signal from a coupled dual loop, and the predicted object is a multi-cycle guidance state (Section 3.5). The second line is model-based control: latent world models that plan or imagine over a learned dynamics [28, 29], and receding-horizon model predictive control that plans, executes, and re-anchors at the true state [30]. VST plays the analogous role at the multi-agent level, with the committed communication trajectory as the observable dynamics; but where Dreamer trusts its imagined rollout for policy learning and MPC re-plans continuously over a numerical model, our forecast is never trusted directly, and verification is a discrete transactional gate with exact rollback.

2.3 Automated alpha mining and financial LLM agents

Formulaic alpha mining searches a combinatorial space of expression trees for return-predictive signals, using reinforcement learning to assemble complementary alpha collections [31], symbolic regression with risk-seeking gradients [32], LLM-assisted discovery [33], agentic frameworks that regularize for decay-resistance and originality [34], and relational deep models for stock ranking [35]. These target the quality of the discovered factors and treat the search as a monolithic optimization; the closest agentic baseline, AlphaAgent [34], couples an LLM agent with AST-level constraints but contributes a regularized generator. Ours is a communication substrate rather than a generator: we restructure the inter-agent channel and add a verified multi-cycle leap on top of an existing miner, treating the search as a multi-agent process whose inter-agent feedback is the dominant serial cost. The two directions are complementary, since our transport is agnostic to the underlying miner and could wrap an RL, symbolic, or LLM-based generator; our experiments use the Qlib platform [36]. Beyond factor mining, LLMs have been applied broadly across finance [37], from domain-specialized language models [38, 39] and time-series forecasting [40] to end-to-end trading via deep RL and multi-agent or multimodal decision systems [41, 42, 43]. These operate at the trading-policy layer; our contribution sits below it, in how the research agents that produce signals communicate and evolve.

2.4 Speculative execution

Speculative decoding accelerates autoregressive generation by drafting several tokens with a cheaper model and verifying them against the target model, preserving the exact output distribution [7]. VST adopts the same verify-then-commit principle but at a different granularity: the "draft" is a predicted multi-cycle guidance state, and "verification" is the actual execution of both miners and their blinded evaluators followed by a layered quality gate. The broader "speculate optimistically, then validate and commit or abort" pattern is not itself new: it is the defining discipline of optimistic concurrency control in databases [44] and of transactional memory in architecture [45], and our commit/rollback provenance (`transaction_id`, `commit_status`) and post-hoc cross-loop consistency check (L2, Section 3.4) are directly in that tradition; the L2 check is a validation phase in the optimistic-concurrency sense. The novelty here is the *unit* of speculation: a multi-cycle guidance transition of two coupled LLM agent loops, verified by re-executing the real miners and blinded evaluators, rather than tokens (speculative decoding) or memory words (transactional memory). A rejected draft costs speculative overhead but never corrupts the committed state, mirroring the safety property of speculative decoding while operating over a stateful, multi-agent transaction rather than a token sequence. The analogy is only partial. Speculative decoding gives an *exactness* guarantee, in that the accelerated sampler reproduces the target model's output distribution. VST gives only a *state-safety* guarantee: a rejected leap rolls back to an exact prior state, while an accepted leap is validated against empirical quality gates rather than proven distributionally identical to the unaccelerated trajectory. Quality preservation is therefore an empirically estimated bound (Equation (2)), not a theorem.

3 The VST Method

3.1 System Setting and Problem Formulation

3.1.1 Dual-loop system substrate

Before developing the structured transport that carries our contribution, we fix the substrate it wraps and the objective it must respect; nothing in this subsection is ours, and we state it only so the typing of C1 and its downstream leap have a precise object to attach to. We adopt, without modification, the dual-loop evolutionary search over formulaic alphas introduced in prior work [1]; we summarize it here only to fix notation, as it is the substrate on which our contributions operate rather than a contribution of this paper. A single *ResearchReportAgent* advises both mining loops and receives structured feedback from both sides. In the *alpha-discovery loop*, an *AlphaMiner* proposes candidate factors and three independent *AlphaEvaluator* instances produce diagnostic reports; their aggregate miner brief drives the next alpha cycle, and their report brief returns to the report agent. In the *evaluation-optimization loop*, an *EvaluationMiner* proposes metric-set and portfolio changes and three independent *FactorMetricsEvaluator* instances analyze the resulting configuration and empirical evidence in the same multi-cycle pattern. The loops are coupled: the evaluation-side metric set scores alpha candidates, while accepted factors become evaluation-side samples.

To state the contribution boundary explicitly: the dual-loop topology, the six evaluators, and the metric/skill libraries that the evaluators evolve are inherited from prior work [1] and used unchanged. Everything this paper claims sits at and above the communication layer, and all of it rests on one load-bearing move: typing that layer (C1). The multi-horizon guidance predictor (C2), the transactional verify–leap controller (C3), and the *separation* of the predictor's skill evolution from the inherited evaluator libraries (C4) are the three capabilities that typing makes possible,

not independent layers. Where C4 discusses evaluator rubrics, it does so only to specify what the predictor must not influence, not to claim the evaluator-evolution mechanism itself.

Data are partitioned into four disjoint temporal splits: a training segment for alpha search, a validation segment for evaluator self-check and top-K selection, a test segment, and a holdout segment. The test and holdout splits play different roles by design, and we keep both rather than collapsing them into one out-of-sample set. The test segment is visible to the meta-review agents: the evaluators may read test-segment outcomes when they diagnose configurations and feed guidance back into the loop, so the test split, while never used for training, is inside the system's optimization horizon and can carry a mild selection-induced leakage. The holdout segment is stronger: no agent, gate, horizon choice, or skill update is ever allowed to touch it, so it is the only partition that is free of every closed-loop decision. Reporting both lets a reader separate "after the training period but still within the system's view" (test) from "fully isolated" (holdout); an effect that survives only on the latter cannot be an artifact of the loop optimizing toward the evaluation segment. All model-selection signals are drawn from the training and validation partitions; the test and holdout partitions are isolated from horizon selection, skill updates, and every gate decision.

3.1.2 Objective and quality constraint

Let S_t be the committed system state at feedback cycle t: the factor pool, evaluation configuration, report memory, and approved skills. Ordinary execution advances $S_t \to S_{t+1}$ by running one full miner–evaluator–report cycle. Our goal is to predict a future *miner-directed guidance state*, use it to advance both loops by more than one cycle, validate the resulting actual outputs, and skip at least two ordinary cycles when validation succeeds. We report the cycle speedup

$$\text{Speedup}_{\text{cycle}} = T_{\text{base}}/T_{\text{leap}}, \tag{1}$$

and the corresponding wall-clock speedup, subject to a quality constraint on the committed leap state relative to the baseline optimum $\theta^\star$,

$$\text{Quality}(\theta_{\text{leap}}) \;\geq\; \text{Quality}(\theta^\star) - \epsilon. \tag{2}$$

Here $\theta^\star$ denotes the state the unaccelerated baseline reaches by running every ordinary feedback cycle to convergence under the same data splits, budget, and stopping rule; it is the reference the accelerated run must match within tolerance ϵ, and it is computed only on development partitions. Equation (2) is treated as an empirical constraint whose thresholds are fixed before final testing; if every trial fails, state evolution matches the baseline and incurs only the measured speculative overhead, which we report rather than claiming a cost-free worst case. With the substrate and objective fixed, the rest of the method follows from a single decision about how these agents communicate, which we take up next.

3.2 Structured Inter-Agent Communication Protocol

The existing A2A implementation persists each round as a JSON object whose top-level namespaces are the operational payloads exchanged between agents; the agent logs retain the corresponding prompt–response trace. We do not replace this operational format. Instead we add a canonical routing wrapper around it, so that every inter-agent transmission becomes

$$m_{i\to j} = \left\langle\, e_{i\to j},\; x_{f(m)},\; p_{i\to j} \,\right\rangle \in \mathcal{S}_{r(i,j)}, \tag{3}$$

where e is a universal envelope, $f(m)$ is the original A2A field path, $x_{f(m)}$ is that field's native string or dictionary payload, and p is transactional provenance. For example, the field path `advise.`

`to_alpha_miner` maps to the argument `rr_advise` of the miner's `propose` call. The wrapper therefore separates routing and commit semantics from task content while retaining the receiver API used by the completed system.

3.2.1 Design principles for A2A compatibility

As anticipated in the Introduction, this protocol is the baseline system's own communication architecture made explicit, not a format invented to justify a leap. It rests on three principles for the record space $\mathcal{S}$. First, payloads are *A2A-compatible*: canonicalization preserves the existing round-file field path and the receiving method argument, so structuring the channel changes no agent's task interface. Second, packets are *causally addressable*: round, cycle, state, and source-artifact identifiers make every feedback transition replayable. Third, packets are *unicast and visibility typed*: each physical packet has exactly one receiver, and a field is delivered only if its source segment and receiver role satisfy the access policy. Together these principles give the system one communication schema while preserving concrete producer, consumer, visibility, and state-transition semantics on every edge.

3.2.2 Three-layer packet: envelope, body, and provenance

The envelope e contains four field groups, realized as a single typed record (`A2AEnvelope`):

- **identity**: `message_id`, `run_id`, `round`;
- **routing**: `sender`, `receiver`, `receiver_arg`;
- **causality**: `parent_state_id`, `cycle`, `sequence`;
- **contract**: `payload_path`, `schema_version`, `visibility`.

The contract selects the schema for one native body $x_{f(m)}$. The identity fields, the parent-state link, and the sequence are generated deterministically by the gateway, because the existing round JSON stores payloads by namespace rather than as standalone messages. The provenance block p links the packet back to its source round file and the producing agent's log, records whether the packet is *observed* or *speculative*, and records whether its transaction commits (`origin`, `transaction_id`, `commit_status`). Validation proceeds in a fixed order: validate the envelope, validate the body against the selected contract, then admit the packet to a transaction.

Table 1 separates the operational role of each layer from its later use by the predictor. The same fields that make ordinary transmission explicit also serve as stable trajectory coordinates, which is the mechanism by which C1 enables C2.

3.2.3 Deterministic unicast routing

Routing is deterministic and unicast. The envelope names exactly one receiver; when a single logical producer result has content for multiple consumers, the gateway materializes one physical packet per destination. Such packets retain a shared parent identifier but may carry different consumer-specific body types. The role-specific contracts are:

- **AlphaMiner** receives two fields: `advise.to_alpha_miner` maps to `rr_advise`, and `alpha_reports.alpha_miner_brief` maps to `failure_feedback`.
- **EvaluationMiner** receives `advise.to_evaluation_miner` as `rr_advise` and `metrics_reports.evaluation_miner_brief` as `evaluator_feedback`.

Table 1: Field-by-field rationale for the structured communication plane. The left column is why a group helps ordinary transmission; the right column is why the same group helps later prediction.

Layer / field group	**Helps transmission**	**Helps prediction**
Envelope: identity	Deterministically identifies each transmission within a persisted round.	Records align into state-indexed trajectory observations Z_t.
Envelope: routing	Receiver and its method argument are explicit.	Unrelated edges stay distinct; inaccessible fields cannot leak into a target.
Envelope: causality	Receivers reconstruct ancestry; the controller can replay a feedback cycle.	The two loops are time-aligned and the future target $t+k$ is well defined.
Envelope: contract	Gateway validates the round-file field against its receiver schema and access policy.	Type identity fixes the decoder, loss, and aggregation rule per field.
Native body $x_{f(m)}$	Retains operational keys (`rr_advise`, `failure_feedback`, `candidates`, `config`).	Prediction targets stay tied to the exact field paths and receiver arguments used at run time.
Provenance p	Evidence is externally inspectable and traceable to its source artifact; speculative state stays isolated until commit.	Only committed prefixes supervise future bundles; rollback traces stay out of the state trajectory.

- Each **AlphaEvaluator** receives `alpha_miner.candidates` and an evaluation summary; the aggregate `alpha_reports.research_report_brief` returns alpha-side findings to the report agent.
- Each **FactorMetricsEvaluator** receives the evaluation configuration, backtest samples, and the visible metric set; the aggregate `metrics_reports.research_report_brief` reaches the report agent only when its visibility policy permits the edge.

Figure 1 shows this hierarchy and the complete route catalogue.

3.2.4 Visibility typing and leakage prevention

The visibility field enforces an access policy that becomes essential once a predictor is added. Prediction packets are admitted only when routed to a miner, only when their field path is one of the four predicted paths, and only under `miners_only` visibility; a packet addressed to an evaluator with a speculative origin is rejected before persistence. This rule is enforced by the gateway's validation step rather than by convention, so the six evaluators structurally cannot receive a forecast. Separating this policy into a typed field, rather than leaving it implicit in prose, is what lets the same channel safely carry both observed and speculative traffic.

3.2.5 Worked example of a canonical packet

The canonical wrapper around one actual advice transmission from the report agent to the AlphaMiner is a single record with an `envelope` block (identity, routing, causality, contract), a `body` carrying `rr_advise`, and a `provenance` block naming the source round file, the producing agent log, an `origin` of `observed`, and a `commit_status` of `committed`. All other edges use the same envelope with a different body schema.

Every edge now carries this structure. That is the whole of the load-bearing contribution; the three capabilities in the sections that follow, forecasting the trajectory, verifiably committing a leap over it, and co-evolving the predictor without corrupting its judges, are what it makes possible, and none of them could be built on a prose channel.

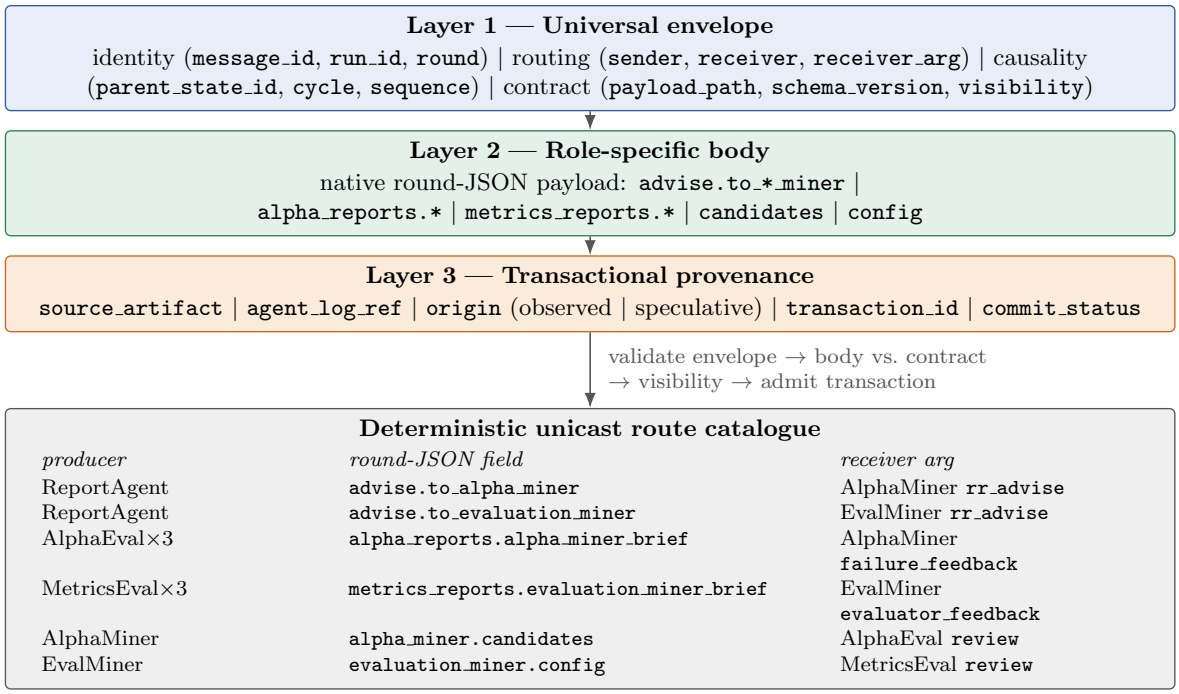


Figure 1: A2A-compatible three-layer protocol and deterministic route catalogue. The wrapper adds identity, routing, causality, visibility, and provenance while retaining the original round-file `payload_path` and receiver argument. Native round-JSON fields are wrapped, routed to existing method arguments, and retained as prediction targets.

3.3 Multi-Horizon Structured Trajectory Prediction

Once C1 makes the channel a typed, replayable record, the committed hand-offs form a trajectory a model can be trained on.

3.3.1 State-aligned communication trajectory

Let Z_t denote the committed, state-aligned communication graph at feedback cycle t. It contains every visibility-legal packet among the report agent, the two miners, both evaluator groups, and the controller: report advice, candidate and configuration packets, evaluator analyses, consumer-specific briefs, evidence references, and the resulting control event. The miner-directed guidance at cycle t is extracted from four exact A2A field paths:

$$\begin{aligned} A_t^{\alpha} &= Z_t[\texttt{advise.to_alpha_miner}], \\ \bar{R}_t^{\alpha} &= Z_t[\texttt{alpha_reports.alpha_miner_brief}], \\ A_t^{\text{eval}} &= Z_t[\texttt{advise.to_evaluation_miner}], \\ \bar{R}_t^{\text{eval}} &= Z_t[\texttt{metrics_reports.evaluation_miner_brief}], \\ G_t^{\alpha} &= (A_t^{\alpha}, \bar{R}_t^{\alpha}), \qquad G_t^{\text{eval}} = (A_t^{\text{eval}}, \bar{R}_t^{\text{eval}}). \end{aligned} \tag{4}$$

The two A channels are the two strings emitted by the single report agent; the two $\bar{R}$ channels are the structured aggregates of the corresponding three evaluator reports. Future report advice is therefore predicted explicitly and delivered directly to the same miner argument as in ordinary execution.

3.3.2 Multi-horizon prediction target

For each candidate horizon $k \in \mathcal{K} = \{2, 4, \ldots, K_{\max}\}$, the supervised target is the accumulated structured guidance that the two miners would receive over the next k ordinary cycles,

$$B_{t,k} = \text{Aggregate}\Big(\{G^{\alpha}_{t+\ell}, G^{\text{eval}}_{t+\ell}\}_{\ell=1}^{k}\Big), \tag{5}$$

where the type-aware aggregator composes set edits, resolves repeated promote/demote decisions by recency and support, and retains the latest bounded categorical adjustment. This is a future *guidance* target, not a future factor, metric, verdict, or score.

The aggregator is type-directed rather than a single reducer: each of the four channels carries fields of known type, and AGGREGATE dispatches per field type (Algorithm 1, Appendix A). Set-valued fields (for example, the accepted metric set or the active factor tags) accumulate additions and removals in cycle order and collapse repeated promote/demote decisions on the same element to the most recent one weighted by how many cycles supported it. Numeric or bounded-categorical fields (for example, a weight or a horizon bias) keep the latest value within the schema's declared range. String advice fields keep the most recent committed text. The type-directed replay makes $B_{t,k}$ a deterministic function of the committed window, so the same prefix always yields the same target.

3.3.3 Predictor architecture and type-aware losses

The predictor reads a sliding window $Z_{t-w+1:t}$ and a requested k, then emits $(\widehat{B}_{t,k}, q_{t,k}, u_{t,k}) = P_\phi(Z_{t-w+1:t}, k)$: a schema-constrained bundle, a calibrated success confidence q, and an epistemic uncertainty u. Categorical, set-valued, and numeric fields use type-appropriate losses,

$$\begin{aligned}\mathcal{L}_P = \lambda_c\, \text{CE}(\widehat{B}^{\text{cat}}, B^{\text{cat}}) + \lambda_s\, \text{SetLoss}(\widehat{B}^{\text{set}}, B^{\text{set}}) \\ + \lambda_n \|\widehat{B}^{\text{num}} - B^{\text{num}}\|_1 + \lambda_{\mathcal{S}}\, \text{Penalty}_{\mathcal{S}_B}(\widehat{B}) \\ + \lambda_q\, \text{Brier}(q, y_{\text{commit}}).\end{aligned} \tag{6}$$

Here CE is cross-entropy on categorical fields, and SetLoss is a soft Jaccard (1 minus intersection-over-union) between the predicted and target element sets, which penalizes both missing and spurious set members without imposing an order. $\text{Penalty}_{\mathcal{S}_B}$ is a schema-validity term that is zero when $\widehat{B}$ satisfies the bundle schema $\mathcal{S}_B$ and positive for each violated constraint (out-of-range numeric, unknown categorical level, illegal route), which trains the heads to emit admissible bundles. The final term is the Brier score of the calibrated confidence q against the realized commit outcome $y_{\text{commit}} \in \{0, 1\}$. The weights $\lambda_c, \lambda_s, \lambda_n, \lambda_{\mathcal{S}}, \lambda_q$ balance the per-type terms and are selected on a development split; the schema-penalty weight $\lambda_{\mathcal{S}}$ is set high enough that admissibility dominates, since an inadmissible bundle is rejected at L1 regardless of its content. The predictor is one component containing an open-weight LLM, a shared trajectory encoder, a trajectory-skill library, an RL-trained horizon policy, and four typed decoding heads,

$$\widehat{B}_{t,k} = \big(\widehat{A}^{\alpha}_{t,k},\ \widehat{\widehat{R}}^{\alpha}_{t,k},\ \widehat{A}^{\text{eval}}_{t,k},\ \widehat{\widehat{R}}^{\text{eval}}_{t,k}\big). \tag{7}$$

In our implementation the open-weight LLM is `gpt-oss-120b`, served locally through an OpenAI-compatible endpoint; it is the only agent that receives a trainable local model, while the miner, evaluator, and report agents keep their existing hosted-API configuration. The base checkpoint is frozen and only a LoRA adapter is updated, first by supervised fine-tuning on causal trajectory prefixes and then by the constrained policy update of Section 3.5. The four channels share one

global state representation because they are causally coupled. In this sense the predictor plays the role of a learned dynamics model over the communication trajectory, analogous to latent world models used for planning [28, 29], except that its forecast is never trusted directly: it is validated by real execution (Section 3.4). The communication layer serializes this logical action as two unicast packets, each addressed directly to its miner and consumed as one `propose` call:

$$\begin{aligned}
\hat{m}^{\alpha}_{t,k} &: \texttt{sender=predictor_agent},\ \texttt{receiver=alpha_miner},\\
&\quad \texttt{rr_advise} \leftarrow \widehat{A}^{\alpha}_{t,k},\ \texttt{failure_feedback} \leftarrow \widehat{\bar{R}}^{\alpha}_{t,k},\\
\hat{m}^{\text{eval}}_{t,k} &: \texttt{sender=predictor_agent},\ \texttt{receiver=evaluation_miner},\\
&\quad \texttt{rr_advise} \leftarrow \widehat{A}^{\text{eval}}_{t,k},\ \texttt{evaluator_feedback} \leftarrow \widehat{\bar{R}}^{\text{eval}}_{t,k}\ .
\end{aligned} \tag{8}$$

Both have `origin=speculative` and the same parent state. No third packet is created for an evaluator or the report agent. Four predicted structures thus require four output heads, but neither four predictor agents nor four independent RL policies.

3.3.4 Training corpus and no-leakage guarantee

The structured gateway already validates every packet, so it also appends the committed packet, prompt, sampled tokens, log probabilities, and state identifier to an immutable trajectory log. Sampled assistant tokens are copied verbatim; only fixed scaffolding is retokenized, and a loss mask marks model tokens as trainable. "Deterministic replay" has a specific meaning here: because the producing agents are LLMs (hosted-API miners and evaluators, and a local predictor model) whose sampling is not reproducible by re-invocation, replay is the deterministic reconstruction of a trajectory *from the recorded log of tokens and states*, not deterministic re-execution of the agents. Likewise, "immutable" refers to an append-only log that is never edited in place after commit; we do not claim tamper-evidence or content-addressed hashing beyond append-only discipline. The target constructor projects the committed graph onto the four state-aligned paths of Equation (4). Training examples pair the same global causal prefix with several direct horizons, so a horizon-eight target is learned from the observed window $t{+}1{:}t{+}8$ rather than generated by chaining eight one-step predictions. This is the direct (as opposed to recursive) multi-step forecasting strategy [46], which avoids compounding one-step error across the horizon; what is specific to our setting is that the predicted object is a typed structured guidance bundle over a communication graph rather than a scalar or vector series. At cycle t, the prefix contains only packets committed by t; controller-only evidence is excluded by the visibility field; the test and holdout partitions never form guidance targets, select a horizon, update a skill, or compute a reward; and rolled-back traces are labeled failures and cannot become positive supervised targets. Figure 2 summarizes the pipeline.

3.4 Transactional Verification and Leap Control

A forecast is only useful if an incorrect one is inexpensive to reverse. Because C1 records provenance and commit status per packet and isolates speculative traffic until it commits, the leap can be run as a transaction: propose the forecast, execute it for real in a scratch state, and either commit the whole transition or discard it with no trace left in the committed trajectory.

Given a predicted bundle $\widehat{B}_{t,k}$, the two miners produce actual outputs $O_t = (O^{\alpha}_t, O^{\text{eval}}_t)$ in a scratch namespace. The six evaluators then produce independent reports $E_t = (E^{\alpha}_{t,1:3}, E^{\text{metrics}}_{t,1:3})$. The controller validates this *speculative outcome*, not whether the miner output matches the predicted guidance field by field. Four levels short-circuit in ascending order of cost.

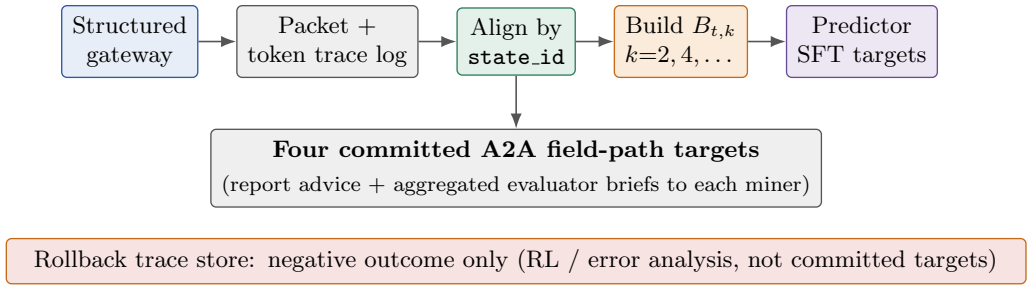


Figure 2: Predictor data pipeline. The structured gateway records committed packets and token-faithful model traces, aligns four miner-directed target channels by state, and composes them into two destination-specific guidance packets at each direct horizon. Rollback traces are stored separately for RL and error analysis, not inserted into the committed supervised trajectory.

3.4.1 Adaptive horizon selection

At state S_t the predictor scores every candidate horizon and the controller chooses the farthest admissible one,

$$k_t = \max\left\{k \in \mathcal{K} : k \geq 2,\ q_{t,k} \geq \tau_q,\ u_{t,k} \leq \tau_u,\ \widehat{G}_{t,k} > 0\right\}, \tag{9}$$

where $\widehat{G}_{t,k}$ is the predicted saved compute after speculative and verification costs. If the set is empty, the system runs one normal feedback cycle and makes no leap. For an admissible k_t, the controller opens a scratch transaction and routes the two payload branches only to their respective miners; the scratch execution may use up to b coupled refinements to resolve cross-loop inconsistency, but a trial counts as a speedup only when $b < k_t$.

3.4.2 Four-level validation gate

L1: Bundle legality and route policy

$$V_1 = \mathbf{1}\left[\widehat{B}_{t,k} \models \mathcal{S}_B \ \wedge\ \text{targets}(\widehat{B}_{t,k}) = \{\texttt{alpha_miner}, \texttt{evaluation_miner}\}\right]. \tag{10}$$

This rejects malformed fields, stale parents, $k < 2$, and any evaluator-targeted prediction before speculative execution.

L2: Actual-output validity and cross-loop consistency

$$V_2 = \mathbf{1}\left[O_t^{\alpha} \models \mathcal{S}_{\alpha} \wedge O_t^{\text{eval}} \models \mathcal{S}_{\text{eval}} \wedge \text{CrossConsistent}(O_t^{\alpha}, O_t^{\text{eval}}, S_t)\right]. \tag{11}$$

The factor batch must be executable, the evaluation configuration legal, and the metric-set, sample, portfolio, and state references must agree. The gate does not require either output to equal a value in the predicted bundle. $\text{CrossConsistent}(O_t^{\alpha}, O_t^{\text{eval}}, S_t)$ returns true only when three referential checks hold: (i) every factor the evaluation side scores exists in the alpha side's produced batch (no dangling reference); (ii) the metric set named by O_t^{eval} is a subset of the metrics admissible at state S_t; and (iii) the portfolio in O_t^{eval} is built only from factors that pass the alpha side's own executability check. The check is an optimistic-concurrency validation phase in the sense of validation-based concurrency control: the two coupled loops execute speculatively and their joint state is checked for conflicts before commit. These three referential checks are necessary conditions we found sufficient to catch the divergences observed in our runs, not a proven-complete characterization of cross-loop consistency; a conflict outside this enumerated set would be admitted,

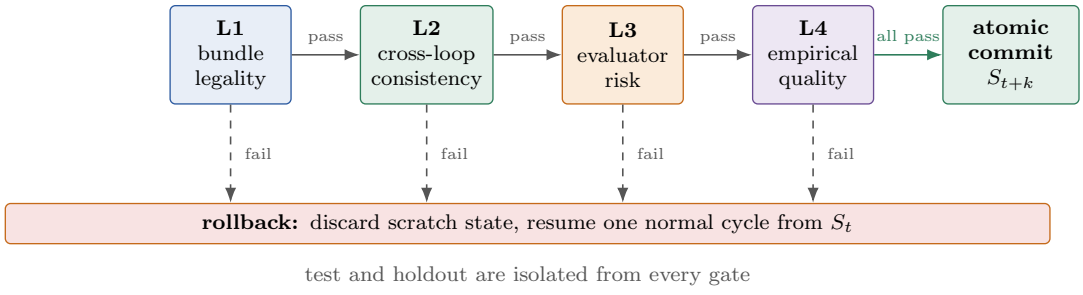


Figure 3: Four-level validation of a speculative outcome. The bundle is checked first; the actual miner outputs, blinded evaluator reports, and development-set evidence are checked next. Only an all-pass path atomically commits; any failure discards the scratch transaction and resumes one normal cycle from S_t.

so the check is a conservative screen rather than a soundness guarantee. Any violation fails L2, because it means the two loops have diverged into an incoherent joint state that no downstream quality score would be meaningful on.

L3: Blinded evaluator risk Each three-instance group writes reports independently before aggregation. The controller maps schema-defined critical findings to robust risk scores while preserving report-level severity and disagreement,

$$V_3 = \mathbf{1}\big[\mathrm{RiskAgg}(E^{\alpha}_{t,1:3}) \le \tau_\alpha \ \wedge\ \mathrm{RiskAgg}(E^{\mathrm{metrics}}_{t,1:3}) \le \tau_m\big]. \tag{12}$$

RiskAgg maps each report's schema-tagged findings to a bounded severity score (a critical finding dominates a major, which dominates a minor) and then aggregates the three independent scores with a disagreement-preserving statistic: it takes the worst-case severity rather than the mean, so a single instance flagging a critical issue is not averaged away by two lenient ones. L3 is thus conservative by construction and keeps the three evaluators' independence meaningful.

L4: Empirical development gate The controller recomputes factor and portfolio quality from the miners' actual outputs on two development partitions, a train gate and an internal leap-validation split,

$$V_4 = \mathbf{1}\Big[\min_d Q_{\alpha,d}(O^{\alpha}_t) \ge \tau_g \ \wedge\ \min_d Q_{m,d}(O_t) \ge \tau_p\Big], \tag{13}$$

with $d \in \{\text{train-gate}, \text{leap-valid}\}$, where $Q_{\alpha,d}$ combines IC/RankIC stability and $Q_{m,d}$ combines portfolio return, risk, monotonicity, and cost. Final test and holdout data are isolated from all adaptive decisions. The transaction decision is

$$C_t = \prod_{\ell=1}^{4} V_\ell, \qquad C_t = 1 \Rightarrow \mathrm{AtomicCommit}, \quad C_t = 0 \Rightarrow \mathrm{Rollback}. \tag{14}$$

Figure 3 shows the short-circuit structure.

3.4.3 Transactional commit and rollback semantics

We use transactional vocabulary (`transaction_id`, `commit_status`, atomic commit, rollback) in a narrowly restricted sense, and state the model explicitly to avoid overclaiming the ACID properties the terms carry in a database. The concurrency model is *single-writer and strictly serial*:

exactly one speculative transaction is in flight at a time, rooted at the current committed state, and the controller is its sole coordinator. Because execution is strictly serial, serializability holds trivially: there is no concurrent schedule to reorder, so isolation is immediate and we make no claim about a concurrency-control protocol. "Atomic commit" means the validated state is installed only after all four gates pass and the commit is applied as a single controller step from the workflow's point of view; we do not claim crash-atomicity or durability (a process failure during the multi-store install is out of scope, not recovered by a write-ahead log), and "exact rollback" is best-effort restoration of the application-level committed stores (factor pool, report memory, evaluator and predictor libraries) to S_t, established by construction over the state we track rather than proved against arbitrary side effects. Under this model a speculative trial is a transaction rooted at `parent_state_id`. Its namespace contains both miner outputs, all six evaluator reports, the report agent's transaction-local synthesis, empirical artifacts, and any proposed changes to the predictor skill library or the evaluator libraries. None is visible in the committed databases or active agent memories before $C_t = 1$. On commit, the controller installs the validated factor and configuration state, sanitized report feedback, and approved skill updates as a single logical step (absent process failure, per the scope above). On rollback, it deletes the entire mutable namespace and resumes one ordinary feedback cycle from S_t; the immutable audit record is retained with `commit_status=rollback` for RL and failure analysis but is excluded from the active factor pool, report memory, evaluator libraries, and positive supervised targets. This preserves an exact state rollback while still learning from failed speculation. A failed trial adds runtime overhead; it does not alter the baseline state transition.

3.4.4 State safety and quality control

Rollback gives deterministic state safety, while the quality of a committed leap depends on gate calibration. We therefore estimate

$$\Pr\left[\,\mathrm{Quality}(S_{t+k}) < \mathrm{Quality}_{\mathrm{base}}(S_t, k) - \epsilon \;\middle|\; C_t = 1\,\right] \le \delta \tag{15}$$

on development runs, with thresholds fixed before final testing. The selected k is a direct target, not a chain of one-step forecasts; a successful horizon-k trial commits the validated scratch result as S_{t+k} and records that k ordinary guidance cycles were replaced by b speculative refinements. The next decision is always anchored at the latest committed state.

3.5 Separated Co-Evolution of Predictor and Evaluator Skills

The predictor improves by learning from outcomes, and it learns safely because the evaluators that judge it never see what it predicted. Both are consequences of C1: the commit/rollback verdict is a typed, replayable outcome the policy can be trained on, and the same visibility typing that keeps forecasts out of evaluator prompts keeps their skill libraries structurally separate.

The predictor is not frozen after supervised training. Its skill library stores reusable procedures for type-aware feedback aggregation, cross-loop consistency, trajectory change detection, horizon calibration, and rollback diagnosis. The RL policy selects and composes these skills when producing $(k_t, \widehat{B}_{t,k})$, and commit or rollback outcomes then provide outcome-grounded supervision as the mining regime changes.

3.5.1 Agentic reinforcement learning of the predictor

The committed trajectories support online policy improvement inside the predictor. Whereas single-agent LLM policies are typically trained to produce an answer or an action sequence [19, 21]

and optimized with policy-gradient methods such as PPO [23] or its group-relative variant [25], our predictor is optimized against a *transaction-level* outcome from a coupled dual loop. The RL state is $Z_{t-w+1:t}$; the joint action is $a_t = (k_t, \widehat{B}_{t,k_t})$, where $\widehat{B}$ contains all four typed channels; and the environment is the transactional dual loop consisting of both miners, six blinded evaluators, the report interaction, and the controller. We use one predictor with a shared policy and four typed heads (rationale below). During supervised initialization each head has a schema-appropriate auxiliary loss; during online RL the complete four-channel action receives one transaction-level reward. Let $C_t \in \{0, 1\}$ be the controller decision, $C_{\text{saved}}(k_t, b)$ the measured cost of the skipped ordinary cycles, and C_{trial} the speculative plus verification cost. We use

$$r_t = C_t\big[C_{\text{saved}}(k_t, b) - C_{\text{trial}}\big] - (1 - C_t)C_{\text{trial}} - \lambda_r R_t - \lambda_q D_t, \tag{16}$$

where R_t is evaluator risk and D_t is any development-set quality deficit. This rewards a long leap only when it both commits and saves real work; rollback traces train the policy to shorten the horizon or change guidance in similar states. Supervised learning fits the bundle content, RL fits horizon and utility, and the skill library supplies reusable reasoning procedures; all three belong to the same predictor.

We claim no new optimizer here; the contribution is two design choices the structured, transactional channel makes available. One is representational: a single shared-encoder policy emits all four typed heads, rather than four RL agents, because the channels are one coupled state and splitting them would turn cross-channel consistency into a non-stationary coordination problem. The other concerns where the reward comes from: the four-level gate serves as the RL environment, so the learning signal is its commit/rollback verdict on the real dual loop rather than a proxy score, and the trajectories it rejects supply the policy's negatives. Neither choice is available on a prose channel, which offers no typed joint action to share and no atomic outcome to reward against.

3.5.2 Skill lifecycle

Predictor skills evolve by three transactional operations, echoing the growing executable-skill library of open-ended agents [27] but gated by transactional outcomes rather than task success alone. *Acquisition*: a recurring feedback-transition pattern with high horizon-conditioned bundle loss can spawn a specialized trajectory skill. *Reinforcement*: a skill gains selection weight only when its bundle causes the actual dual loop to pass L1–L4 and produce positive net cycle savings. *Pruning or repair*: rollback reason, evaluator risk, and field-level bundle residuals identify whether to shorten a skill's horizon, constrain its output, or rewrite it. Updates occur after the outcome is known, never during scoring.

3.5.3 Separation of predictor and evaluator skill libraries

The six evaluators do not use predicted guidance. Each AlphaEvaluator receives only the actual candidate batch, its development-side evaluation summary, the allowed metric dimensions, and its assigned historical view; it writes an independent report and may propose a new or revised rubric. Each FactorMetricsEvaluator receives only actual factor and backtest samples, portfolio output, the metric set, and its assigned historical view; it may propose a meta-rubric. The three reports on each side are aggregated into distinct miner and report-agent briefs while preserving report-level severity and disagreement. Evaluator proposals are transaction-local while the controller is deciding: if the leap commits, proposals supported by the verified evidence enter the normal trial-to-promotion process; if it rolls back, the proposals are discarded with the scratch state, and the labeled actual-output trace remains available as a negative example for later rubric repair. Predictor identifiers,

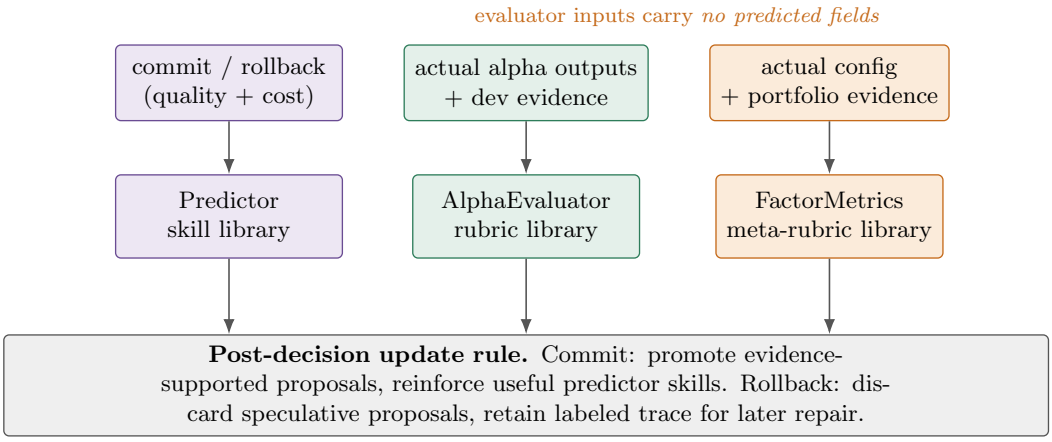


Figure 4: Three transactional evolution paths. Predictor skills learn from commit/rollback outcomes. Evaluators evolve rubric and meta-rubric libraries from actual outputs only, never seeing the prediction, and their updates install only through the post-decision promotion path.

k, confidence, uncertainty, and predicted fields are absent from every evaluator prompt, which prevents forecast anchoring.

Keeping these update paths separate is a safety property with a specific failure it prevents. Predictor skills optimize forecasting and horizon choice; evaluator rubrics optimize diagnosis of actual outputs. If a single successful predictor could also shape the evaluator rubrics, it could gradually teach the evaluators to agree with its own forecasts, and the L3 gate would lose its independence. This is the reward-hacking hazard of a coupled learner and its own judge [47]: an optimizer with influence over its evaluation criterion tends to game the criterion rather than the underlying objective. The transactional promotion rule additionally keeps speculative failures from silently changing active evaluation standards. Figure 4 shows the three separate evolution paths.

4 Experiments

Our claims are of two kinds, and the experiments separate them. The comparison against baselines (Sections 4.2–4.6) asks whether the verified structured search yields factors that survive out of sample; the transport ablation (Section 4.7) asks the load-bearing question directly, whether the typing of C1 earns its place through predictive accuracy or, as we argue, through the correctness properties that C2–C4 depend on. The second question is the one that decides whether structure, rather than speed or returns, is the contribution, and we keep it in view throughout.

4.1 Setup: Universe, Splits, and Baselines

We evaluate on the CSI 1000 (ZZ1000) constituent universe with disjoint temporal splits for training, validation, test, and holdout; the holdout partition is never touched by the closed loop and is used only for final out-of-sample reporting. Every method mines factors on the training split and is filtered to valid candidates on the validation split under a common gate (Sharpe ≥ 1 and annualized excess return ≥ 0.05); no top-K post-selection is applied. All reported quality figures are computed with a single shared 10-group backtest evaluator so that cross-method numbers are directly comparable. All return and risk-adjusted figures reported in this paper are *gross*, before transaction costs, slippage, and turnover constraints; because every method is scored under the

identical shared backtest, the comparison is internally consistent, but the absolute magnitudes are not net-of-cost and should not be read as deployable returns. On a mid- and small-cap universe such as CSI 1000, turnover and cost adjustment can be material; we treat a net-of-cost evaluation as out of scope for this study and note it as a limitation (Section 6).

We compare against seven baselines spanning hand-crafted, symbolic, reinforcement-learning, and LLM-agent alpha discovery: Alpha101 (B1) [48], a genetic-programming searcher (B2, the gplearn baseline from the AlphaGen repository [31]), DSO/DSR (B3) [32], AlphaGen (B4) [31], AlphaMCTS (B5) [49], AlphaAgent (B6) [34], and an automated Alpha-GPT approximation (B7) [50]. Our system is denoted OURS (M4). We report, per method and per split, the number of valid factors n_{valid}, the fraction of factors with backtest Sharpe above 1 and above 2, and the median Sharpe. To compare per-factor Sharpe distributions between OURS and each baseline we use the Mann–Whitney U statistic, which we summarize as the *probability of superiority* $\hat{P}(\text{OURS} > \text{baseline}) = U/(n_1 n_2)$ (the probability that a uniformly drawn OURS factor has larger Sharpe than a uniformly drawn baseline factor). We report this rank effect size rather than a tail p-value: because factors produced within one run share a training window and the same closed-loop selection, they are not independent, so the effective sample size is far below n_{valid} and an i.i.d. p-value would be misleadingly small (orders of magnitude too significant). The probability of superiority is a descriptive rank statistic on a single committed pool per method that does not presuppose independence; it is not a test over independent runs, and between-run variance is not bounded here (see Section 6).

The valid-factor counts are matched across methods by budget rather than left to differ by run length. Each method contributes the valid factors it accumulates in mining order up to a common cap of 500; the baselines emit a one-shot batch at their configured budget (here ≤ 500, with B4 AlphaGen reaching only 200 and B1 providing the 14 hand-crafted Alpha101 formulas), and for our continual system we take the first 500 valid factors of the run in mining order, matching the baselines' "accumulate until the budget is reached" rule rather than selecting the Sharpe-best 500. Because the per-factor quality metrics (median Sharpe, $P(\text{sh}{>}1)$) and the rank-based probability of superiority are computed per factor and are not normalized by count, this cap removes any concern that a larger n inflates them; we report n explicitly so the reader can weigh each comparison. We are explicit about one property of this particular matching rule: for our continual system the first 500 valid factors are not a representative sample of the whole run. That early segment has a higher holdout median Sharpe (+0.34) than the run as a whole (+0.13 over all 2300 scored factors) or its later segments ($\approx +0.10$), so "first in mining order" happens to select the stronger part of our pool, whereas each baseline contributes its entire one-shot batch. The matched-budget median in Table 2 should therefore be read together with the full-pool median (+0.13), and the load-bearing budget-controlled comparison is the *order-agnostic* top-K-by-validation-Sharpe result of Section 4.4, which ranks each method by an identical rule and does not depend on mining order; our out-of-sample edge persists there at $K = 100$ and $K = 200$. The baselines were run under their own recommended search budgets and configurations rather than weakened: that they reach strong in-sample fit supports this, since on the validation split several sit between 1.5 and 1.8 median Sharpe (for example B5 AlphaMCTS at 1.838; Table 5). Their out-of-sample collapse is therefore a robustness gap, not an artifact of under-tuning.

All data splits, the common validity gate, and every gate threshold are defined by a fixed experiment configuration (`config_formal_v1.yaml`) committed before final testing; the test and holdout partitions are read only after these values are frozen, so no threshold or horizon choice is informed by out-of-sample data.

4.2 Out-of-sample comparison

Table 2 reports the holdout split, the strictest out-of-sample test. What matters most for a trading application is realized return, which we report first: on holdout, our system is the only method with a positive median annualized return (+0.059); every baseline loses money out of sample, including the strongest learned ones (B4 AlphaGen −0.001, B5 AlphaMCTS −0.015, B7 AlphaGPT −0.031). The same ordering holds on a risk-adjusted basis: our system attains a median Sharpe of 0.346, again the only positive median, while every baseline degrades into negative territory (B4 −0.006, B5 −0.056, B7 −0.159). OURS also attains the highest fraction of high-quality factors, with $P(\text{Sharpe}>1) = 0.313$ (versus 0.224 for the next-best baseline, Table 2) and, on the stricter $P(\text{Sharpe}>2) = 0.114$, again the highest of any method. The rank effect size points the same way: the probability that a uniformly drawn OURS factor has higher holdout Sharpe than a uniformly drawn baseline factor is above 0.5 against every baseline, though the margin is small for the strongest learned baselines and only large against the fully collapsed ones ($\hat{P}(\text{OURS}>\text{base}) = 0.56$ vs. B4, 0.58 vs. B5, 0.57 vs. B6, 0.57 vs. B7, 0.66 vs. B2, and 0.87 vs. B3; 0.64 vs. B1 on only 14 factors). We report these as descriptive rank statistics, not powered tests: per-factor units within a run are not independent, so we do not attach p-values (Section 4.1). The measured result is that OURS holds a consistent but small per-factor rank advantage over the competitive learned baselines (B4–B7) and a large one only where the baseline has collapsed out of sample.

The positive figure is an *absolute* annualized return; the median *excess* return over the CSI 1000 benchmark remains negative for every method, including ours (−0.028), though ours is the smallest shortfall. The precise statement is therefore that our system is the only one to produce a positive absolute out-of-sample return and the one whose excess return decays least, not that it beats the benchmark on holdout.

The 0.346 median is moderate in absolute terms; the claim is comparative. Under the shared backtest, ours is the only method that stays positive out of sample, partly because the learned baselines overfit into negative territory rather than because our factors are strong in absolute terms.

To quantify the sampling uncertainty of these medians we compute a 95% bootstrap confidence interval by resampling each method's holdout factor pool (2000 resamples, fixed seed). Our system's interval is $[+0.207, +0.508]$, entirely above zero, whereas no baseline has a positive lower bound: several sit entirely below zero (B2 $[-0.644, -0.524]$, B3 $[-1.706, -1.648]$) and the strongest learned baselines straddle zero (B4 $[-0.317, +0.127]$, B5 $[-0.296, +0.062]$). The positive-median result for our system is therefore not an artifact of pool sampling. This interval bounds the sampling variability of the median within a single committed factor pool; it does not bound variability across independent training runs, which would require repeated end-to-end runs and remains a limitation (Section 6). For the same reason, that our interval and a baseline's interval do not overlap is not a between-method significance test: both intervals hold their respective single runs fixed and exclude the dominant run-to-run variance component, so non-overlap shows only that each single pool's median is stably estimated, not that the two underlying search processes differ.

4.3 Portfolio-level out-of-sample performance

The per-factor medians above characterize the *pool* our system produces, but a tradable strategy is a *portfolio*, so we also report portfolio-level holdout performance. Our system commits 852 composite portfolios over its run. To avoid any out-of-sample leakage, we select portfolios only on development data (ranking by test-split Sharpe) and then record the holdout performance of the selected set; holdout is never used for selection. Table 3 reports the development-selected

Table 2: Holdout comparison (CSI 1000, shared 10-group backtest; each method mined on train, filtered to valid on validation; no top-K post-selection). n_{valid} is the validation-filtered pool size under a matched budget: every method contributes the valid factors it accumulates in mining order up to a common cap of 500 (B4 AlphaGen reaches only 200 and B1 provides 14 hand-crafted formulas, reported honestly). For OURS this is the first 500 valid factors of the continual run in mining order, not a Sharpe-ranked selection; 499 of them produced a scored holdout backtest (one dropped for missing holdout coverage), and the holdout statistics are computed over those 499. This early segment is stronger than the full run (full-pool median Sharpe +0.13; see Section 4.1), so this row should be read with the order-agnostic top-K comparison of Table 4. *B7 is an automated Alpha-GPT approximation. The 95% CI is a bootstrap interval for the median Sharpe (2000 resamples of each holdout pool); only our system's interval lies entirely above zero, which bounds within-pool median stability and is not a between-method test (Section 4.2). The per-factor rank effect size $\hat{P}$(OURS>baseline) is reported in text; per-factor units within a run are not independent, so we give a descriptive rank statistic rather than a p-value (Section 4.1).

Method	n_{valid}	P(sh>1)	med. AnnRet	med. sh	95% CI (sh)
B1 Alpha101 [48]	14	0.000	−0.078	−0.396	[−0.91, +0.51]
B2 GP (gplearn) [31]	500	0.116	−0.144	−0.593	[−0.64, −0.52]
B3 DSO/DSR [32]	500	0.010	−0.287	−1.648	[−1.71, −1.65]
B4 AlphaGen [31]	200	0.188	−0.001	−0.006	[−0.32, +0.13]
B5 AlphaMCTS [49]	500	0.160	−0.015	−0.056	[−0.30, +0.06]
B6 AlphaAgent [34]	500	0.220	−0.005	−0.021	[−0.21, +0.11]
B7 Alpha-GPT* [50]	500	0.224	−0.031	−0.159	[−0.32, +0.01]
OURS (M4)	**500** (499)	**0.313**	**+0.059**	**0.346**	[**+0.21**, **+0.51**]

top-5/10/20 portfolios.

The selected portfolios are profitable out of sample on a gross basis: the development-top-20 set has a median holdout Sharpe of +0.71 and a median holdout annualized return of +13.3%, with 80% of the selected portfolios positive and 45% above a Sharpe of one. Two caveats attach directly to this figure. First, it is gross of transaction costs, slippage, and turnover; on a mid- and small-cap universe such as CSI 1000, where spreads, market impact, and a sell-side stamp duty are material, a gross Sharpe of this magnitude can erode substantially net of cost, and we report no turnover figure here (a turnover- and cost-adjusted portfolio evaluation is left to future work, Section 6). Second, selection ranks by test-split Sharpe, and the test split lies inside the system's optimization horizon (Section 3.1.1), so it can carry mild selection-induced leakage; the +0.71 therefore reflects selection on a semi-in-horizon split read out on the clean holdout, not a fully out-of-sample selection pipeline. With those caveats, this is the level at which the system is meant to be used (one deploys a chosen portfolio, not the full undifferentiated factor pool), and it is substantially stronger than the per-factor median of Table 2. We report a set of top portfolios rather than a single best one: the single development-best portfolio reaches only +0.25 holdout Sharpe, so the strength is in the selected group, not in a single fortunate selection. We also state the boundary plainly: across all 852 committed portfolios the median holdout Sharpe is negative (−0.23); the positive result is a property of development-guided selection, which is the intended use, not of the raw portfolio population.

Because the entire portfolio-level claim rests on selection, we test whether the development ranking predicts holdout behavior rather than selecting effective portfolios by chance. Over all 852 portfolios, the Spearman rank correlation between development-split (test) Sharpe and holdout Sharpe is +0.19 (Pearson +0.21): positive but weak. The effect is directionally clear in the tails: the

Table 3: Portfolio-level holdout performance of development-selected portfolios (ranked by test-split Sharpe; holdout never used for selection). The selected set is profitable out of sample; the all-portfolio population median is negative, so the effect is due to development-guided selection.

Selection	med. Sharpe	med. AnnRet	P(sh>0)	P(sh>1)
dev-top 5	+0.26	+6.6%	0.80	0.20
dev-top 10	+0.48	+10.8%	0.80	0.40
dev-top 20	**+0.71**	**+13.3%**	**0.80**	**0.45**
all 852 (reference)	−0.23	−4.8%	0.42	0.13

development-top-quartile portfolios reach a holdout median Sharpe of +0.17 while the development-bottom-quartile portfolios sit at −0.43, so the selection signal is real, not noise, but it is well short of a sharp separator. We therefore make the modest claim that development ranking is a useful but imperfect filter: it reliably pushes the selected set toward the positive tail (hence the +0.71 median for the top-20), without our claiming to identify the single best portfolio in advance. We rank by test-split Sharpe because portfolio-level frozen-shadow records were retained only for the test and holdout segments; a purely validation-based selection would be marginally stricter but is not available in our logs.

We flag one further hazard that our within-pool bootstrap does not address. The +0.71 figure is the median of a set chosen as the top-20 of 852 candidate portfolios, and reporting the Sharpe of a selected maximum over many trials is exactly the setting the backtest-overfitting literature warns against: a selected Sharpe is inflated relative to its out-of-sample expectation by the breadth of the search, an effect quantified by the deflated Sharpe ratio and the probability of backtest overfitting [51], the multiple-testing threshold for factor discovery [52], and data-snooping reality checks [53]. Our bootstrap interval bounds within-pool sampling variability but does not deflate for this selection breadth, so the +0.71 is best interpreted as a selected-maximum estimate that a deflated-Sharpe adjustment would discount; the holdout evaluation of the selected set guards against look-ahead but not against selection inflation. We report the all-852 population median (−0.23) alongside it precisely so the reader can see the size of the selection effect.

We quantify the selection benchmark directly. Following the deflated-Sharpe construction [51], the expected maximum Sharpe obtainable by chance when selecting the best of N trials is $\mathrm{SR}_0 \approx \sigma_{\mathrm{SR}}\,\mathbb{E}[\max_N Z]$, where σ_{SR} is the cross-trial standard deviation of the Sharpe estimates and $\mathbb{E}[\max_N Z]$ is the expected maximum of N standard normals, which the same reference approximates as $(1-\gamma)\,\Phi^{-1}(1-\frac{1}{N}) + \gamma\,\Phi^{-1}(1-\frac{1}{Ne})$ with γ the Euler–Mascheroni constant. For our N=852 candidate portfolios, whose development-split Sharpe has standard deviation $\sigma_{\mathrm{SR}} = 0.42$, this gives $\mathbb{E}[\max_N Z] \approx 3.21$ and hence $\mathrm{SR}_0 \approx 0.42 \times 3.21 \approx 1.36$ on the development scale. Two facts then bound the selection concern. First, the twenty selected portfolios all have development Sharpe between 1.50 and 1.98, above SR_0, and only 47 of the 852 trials clear that benchmark, so the selected set is not what pure-noise selection over 852 trials would surface. Second, and more importantly, the +0.71 is measured on the holdout split, which selection never touches: were the selected Sharpe purely an inflation artifact, it would regress toward the population on the independent split, yet 16 of the 20 selected portfolios remain positive on holdout while the all-852 holdout median is −0.23. We stop short of a fully deflated Sharpe with a non-normality correction, because our per-round records store summary Sharpe rather than the return series needed for its skewness and kurtosis terms; and the benchmark treats the trials as independent, whereas the 852 portfolios are correlated, so SR_0 is an upper bound on the effective-independent-trials benchmark. It is therefore a selection-only bound, not the complete adjustment, and we report it as such.

Table 4: Same-footing comparison: each method's top-K factors by validation Sharpe, evaluated on holdout. Median holdout Sharpe (and $P(\text{sh}>1)$). Best median per column in bold. At $K = 50$ the genetic-programming baseline's few strongest factors exceed ours on the median; the ordering reverses and our advantage widens as K grows.

	$K = 50$		$K = 100$		$K = 200$	
Method	med. sh	$P(\text{sh}>1)$	med. sh	$P(\text{sh}>1)$	med. sh	$P(\text{sh}>1)$
B2 GP (gplearn)	**+0.576**	0.320	+0.423	0.270	−0.369	0.170
B3 DSO/DSR	−0.816	0.080	−1.199	0.040	−1.635	0.025
B4 AlphaGen	+0.205	0.184	−0.069	0.186	−0.006	0.188
B5 AlphaMCTS	+0.153	0.020	+0.251	0.090	+0.236	0.140
B6 AlphaAgent	+0.001	0.180	−0.033	0.190	−0.001	0.215
B7 Alpha-GPT	−0.830	0.100	−0.773	0.130	−0.491	0.155
OURS (M4)	+0.493	**0.388**	**+0.406**	**0.374**	**+0.282**	**0.352**

4.4 Same-footing comparison at matched budget

Table 2 already matches the pool size across methods by taking the first 500 valid factors of each run in mining order. A reader may still ask whether that particular selection rule, first-in-mining-order, unduly favors our continual system, or whether the remaining count differences (B1 at 14, B4 at 200) drive the result. We therefore repeat the comparison under a second, stricter matching: for each method we rank its factors by validation-split Sharpe, take the top K, and report how those same top-K factors behave on the untouched holdout split. This selects each method's own best K factors under an identical rule, removing both the pool-size and the selection-order difference. Table 4 reports $K = 50$, $K = 100$, and $K = 200$ (Alpha101 has only 14 factors and is omitted; AlphaGen has 197 and contributes 197 at $K = 200$).

Under matched budget our system attains the highest median holdout Sharpe at $K = 100$ (+0.406 versus +0.423 for GP and +0.251 for the next learned baseline) and $K = 200$ (+0.282 versus +0.236), and the highest fraction of factors with holdout Sharpe above one at every K. We also report a countervailing observation: at the smallest budget ($K = 50$) the genetic-programming baseline is marginally higher in median (+0.576 versus our +0.493), because its few strongest factors remain robust out of sample; that advantage disappears as K grows, whereas ours persists. The matched-budget result shows the out-of-sample edge is not an artifact of pool size.

4.5 Robustness across splits

The advantage is a robustness effect, not a training-fit effect. Table 5 tracks the median Sharpe of each method across the validation, test, and holdout splits. On validation every method performs strongly, several baselines near or above 1.5 and OURS highest at 2.234; the split is inside each method's development horizon, so a high value there certifies fit, not generalization. The ordering is exposed out of sample: as the split moves from validation to holdout, the learned baselines lose nearly all of their edge (B5 falls from 1.838 to −0.056 and B2 from 1.520 to −0.593), whereas OURS degrades substantially less, from 2.234 to 0.346, and is the only method that remains positive on holdout. The decisive quantity is the degradation ratio: every baseline crosses into negative territory while OURS retains a positive risk-adjusted return, which indicates that the factors surfaced by our verified, structured search generalize out of sample rather than overfitting the development period. This robustness is a property of the search process rather than of any single factor. It cannot be attributed to the leap machinery alone. The inherited dual-loop substrate (Section 3.1.1) is part of the same process, and without a leap-disabled control at matched budget we cannot isolate

Table 5: Median backtest Sharpe across splits. OURS shows the smallest out-of-sample degradation and is the only method positive on holdout.

Method	valid	test	holdout
B1 Alpha101	−1.317	−0.385	−0.396
B2 GP (gplearn)	1.520	0.677	−0.593
B3 DSO/DSR	1.482	0.673	−1.648
B4 AlphaGen	1.457	0.509	−0.006
B5 AlphaMCTS	1.838	0.857	−0.056
B6 AlphaAgent	1.531	0.455	−0.021
B7 Alpha-GPT*	1.646	0.469	−0.159
OURS (M4)	2.234	0.467	**0.346**

the contribution of C1–C3 from that of the substrate. The result is therefore consistent with the verified, structured search generalizing better than the baselines, but we leave the isolating control to future work (Section 6).

4.6 Beyond Sharpe: multi-metric comparison

Sharpe alone could mask a metric-specific effect, so Table 6 reports the matched-budget median (the same 500-factor pools as Table 2) of six complementary metrics on both the test and the holdout splits: annualized return (AnnRet), excess annualized return (ExcRet), the information coefficient (IC), its risk-adjusted form (ICIR), and the information ratio (IR), alongside Sharpe. The picture is consistent with the Sharpe result but carries more detail, and we report it in full rather than selecting favorable rows. We report medians for these auxiliary metrics rather than rank effect sizes; the primary endpoint we designated before reading the holdout split is per-factor Sharpe (Table 2), and we avoid multiplying comparisons across correlated metrics.

On holdout, our system leads on the return and risk-adjusted family: the only positive median Sharpe (+0.346), the only positive median annualized return, and the best information ratio (−0.333). It does not lead on raw predictive correlation: its holdout IC (+0.028) and ICIR (+0.335) are lower than AlphaGen's (+0.037, +0.356) and AlphaMCTS's (+0.048, +0.413). On the test split our system trails the strongest learned baselines on Sharpe. Our contribution does not produce factors with the strongest raw signal correlation, and it is not the strongest at the intermediate test split; its distinctive property is that its factors retain positive risk-adjusted return on the strictest out-of-sample split where every baseline turns negative. That is a robustness claim about the search process, not a claim of superior per-factor predictive power. The test-versus-holdout gap is itself informative: the test segment lies inside the meta-review's optimization horizon (Section 3.1.1), so a method that implicitly tunes toward it can appear strong there; the reversal on the fully isolated holdout is what distinguishes genuine out-of-sample robustness from in-horizon fit.

Two apparent tensions in this table resolve cleanly. First, a lower IC alongside a higher Sharpe is not a contradiction: IC measures the cross-sectional rank correlation between a factor and future returns, whereas Sharpe and the information ratio measure the risk-adjusted return of the resulting position. A factor with moderate raw correlation but stable, low-drawdown behavior out of sample can carry a higher Sharpe than a factor with stronger but more volatile correlation. Our result is exactly this shape: our factors are not the strongest signals by IC, but their risk-adjusted return survives the holdout. Second, a skeptic might attribute our positive holdout Sharpe to scale rather than robustness: a continual run produces many candidates, so with a large pool there are more chances to include out-of-sample survivors, and the lower per-factor IC is consistent with individually weaker signals. Two controls address this. The holdout comparison of Table 2 already

Table 6: Matched-budget median of six metrics on test and holdout (the same 500-factor pools as Table 2). Our system leads the return/risk-adjusted family on holdout but not the correlation family (IC/ICIR), and trails on test; we report all metrics on both splits. Best holdout value per metric in bold.

	Sharpe	AnnRet	ExcRet	IC	ICIR	IR
Test split						
OURS	+0.467	+0.055	−0.029	+0.016	+0.190	−0.432
B4	+0.509	+0.083	−0.014	+0.031	+0.234	−0.201
B5	+0.857	+0.158	+0.006	+0.042	+0.380	+0.086
B2	+0.677	+0.119	−0.016	+0.040	+0.288	−0.178
Holdout split						
OURS	**+0.346**	**+0.059**	**−0.028**	+0.028	+0.335	**−0.333**
B4	−0.006	−0.001	−0.043	+0.037	+0.356	−0.400
B5	−0.056	−0.015	−0.061	**+0.048**	**+0.413**	−0.596
B2	−0.593	−0.144	−0.127	+0.035	+0.271	−1.171
B6	−0.021	−0.004	−0.092	+0.014	+0.139	−0.947
B7	−0.159	−0.031	−0.118	+0.011	+0.104	−1.155

caps every method, ours included, at the same 500-factor budget taken in mining order, so its positive median is not a large-n effect; and the matched top-K comparison of Section 4.4 reduces every method to its own best K factors under an identical rule, where our median holdout Sharpe remains highest at $K = 100$ and $K = 200$. The edge is therefore not merely a consequence of producing more candidates: we do not claim stronger per-factor predictive power, only that the search process yields a pool whose risk-adjusted return is more robust out of sample under budget control.

4.7 Ablation: structured versus free-text transport

This ablation establishes the paper's central claim about structured transport: it provides a verification capability that a free-text channel cannot express, at no cost to predictor accuracy. The favorable side is structural. Only the typed arm can enforce the ninth verification check, `prediction_routes`, which guarantees a speculative forecast never reaches an evaluator (detailed below); and the four-level gate discriminates on outcome quality rather than approving indiscriminately (the rejection-level breakdown in the verified-leap analysis of Section 4.7 shows rollbacks concentrated at the semantic gates L3 and L4, never at the format levels). The cost side is a deliberate null: an equal-information free-text channel reaches the same predictor hit rate, so we do not overclaim an accuracy gain. The scope of "no cost" is narrow: the two arms are indistinguishable on predictor hit rate, and on factor quality the same-footing controls below show the free-text arm is if anything slightly ahead, so typing does not improve the factors either. What the typed arm provides is the correctness guarantee, not a quality gain, and that is the contribution we defend.

The design isolates the effect of structured transport: the only manipulated variable is the representation fed to the predictor. The full system (S-full, our M4 run, 1646 rounds) serializes the state as typed structured records, whereas the ablated arm (S-off, 1100 rounds) serializes the same information as an equal-content free-text state. Everything else, namely the predictor model, the verification gate, the data, and the thresholds, is held fixed, and the predictor hit rate is recomputed on a common yardstick that removes the structured-only route-validation check so that both arms are scored identically. Because the two arms differ in run length (1646 versus 1100 rounds), we treat the nested same-footing controls of Table 8 (matched rounds, matched top-K, matched factor count) as the primary evidence for the effectiveness comparison, and read Table 7 as an unmatched-run summary of the predictor-side metrics that motivates it.

Predictor effectiveness and verification scope. The central finding is the *absence of a measurable accuracy advantage* for structured transport. Table 7 shows that in this single paired run the two representations are indistinguishable on the downstream metrics that the predictor optimizes: the committed-leap hit rate is 0.975 for both arms (0.9749 free-text versus 0.9754 typed), the mean committed horizon differs only marginally (2.37 versus 2.47), and the per-round LLM-call cost is effectively identical (10.80 versus 10.89). Because each arm is a single long run under a fixed seed and we do not estimate run-to-run variance, we report this as *no evidence of an accuracy advantage for the typed form*, not as a proof of equivalence; distinguishing a true null from an underpowered one would require replication or a pre-registered equivalence test with a stated margin, neither of which we ran. An offline input-representation probe, which measures the token-level alignment between each representation and the future four-field target over 4978 causal pairs, is consistent with this reading and likewise finds no advantage for the typed form (0.0528 ± 0.0053 typed versus 0.0547 ± 0.0053 free-text; difference within one standard deviation).

What structured transport provides is orthogonal to predictive performance. It is the typed contract, deterministic replayability, and visibility-typed leakage prevention analyzed in Section 3.2. The free-text arm attains the same hit rate but cannot offer any of these: its state cannot be validated against a schema, cannot be replayed as a causal trajectory, and cannot structurally guarantee that a speculative forecast never reaches an evaluator. The contribution of C1 is therefore correctness and auditability at *no cost* to downstream effectiveness, not a gain in predictive accuracy. This reading is the one we carry throughout the paper, and it is what makes the verified leap of Section 3.4 safe rather than merely fast.

The difference is evident in what the controller is able to verify. Each committed transaction records the set of verification checks it passed. These checks are the fine-grained decomposition of the four-level gate of Section 3.4 together with the horizon confidence gate of Equation (9). The free-text arm carries eight checks (minimum horizon, exact four fields, confidence gate, both miner outputs, both aggregated evaluator reports, and empirical quality), all of which operate on the *actual* outputs and are therefore available to either representation. The typed arm carries a ninth, `prediction_routes`, which checks that every speculative packet is addressed only to a miner and never to an evaluator. This check exists only because structured transport makes routing a typed, machine-checkable field: a free-text state has no packet whose destination can be validated, so the leakage guarantee cannot be expressed as a gate at all and would have to rest on convention. Structured transport thus supplies an additional verification condition that the free-text channel is structurally unable to offer. When we equate the two arms' hit rates earlier, we exclude `prediction_routes` from the count so that both are scored on the eight common checks; the excluded check is precisely the safety property that C1 uniquely enables and that the equal-footing hit-rate comparison sets aside.

One further caveat: the offline probe is a deterministic similarity proxy, not a model-in-the-loop measurement; it corroborates the hit-rate reading rather than standing on its own.

Same-footing controls. To rule out that the null is an artifact of unequal run length or pool size, we also compare the two arms' *in-database factor quality* on holdout under three nested same-footing controls. First, we truncate the structured arm to the free-text arm's round count (≤ 1100). Second, on that same-round base we take each arm's top-K factors by their development-split Sharpe. Third, we match the factor count by deterministic subsampling to $\min(n)$. Table 8 reports the outcome: at the full same-round and equal-count pools the two arms are close (both near +0.10 to +0.18 median holdout Sharpe), and when the pools are narrowed to each arm's development-ranked top-K the free-text arm is clearly ahead, not behind (+0.86 versus +0.27 at top-100, +0.80

Table 7: Structured vs. free-text transport ablation (only the predictor's input representation differs; all else fixed). In this single paired run the two arms show no measurable difference on any predictive-effectiveness metric; we read this as no evidence of an accuracy advantage rather than as proven equivalence (single seed, no variance estimate). The value of structured transport is the correctness and auditability properties of Section 3.2, which the free-text arm cannot provide at any hit rate.

Metric	S-off (free-text)	S-full (typed)
Committed-leap hit rate	0.9749	0.9754
Mean committed horizon	2.37	2.47
LLM calls / round	10.80	10.89
Input-repr. alignment (offline)	0.0547±0.0053	0.0528±0.0053
Typed contract / schema check	no	**yes**
Deterministic replay	no	**yes**
Visibility leakage prevention	no	**yes**

Table 8: Ablation under nested same-footing controls (median holdout Sharpe of in-database factors, real frozen-holdout backtests). The arms are close on the full same-round and equal-count pools; on development-ranked top-K the free-text arm is clearly ahead. Structured transport never leads on factor quality at matched rounds, matched top-K, or matched count.

Control (holdout median Sharpe)	S-off (free-text)	S-full (typed)
Same-round base (≤ 1100)	+0.171	+0.098
Top-100 (same-round, dev-ranked)	+0.864	+0.274
Top-200 (same-round, dev-ranked)	+0.797	+0.221
Equal-count (n=1608)	+0.176	+0.098

versus +0.22 at top-200). In no control does structured transport lead. This does not favor the typed arm on factor quality at any matched budget or sample size, which is exactly what the paper claims: the value of structured transport lies in the correctness properties of Section 3.2, not in the factors produced, and if anything the typed arm is mildly behind on this axis.

Runtime and success-rate side effects. Structured transport also moves several operational quantities, all reported in full in Appendix B. In brief, and in the typed arm's favor, the structured arm has a lower median per-round wall-clock (353 s versus 385 s, observational) and a marginally higher portfolio structure-gate pass rate and composite Sharpe; the free-text arm has a higher single-factor evaluator pass rate. None of these is load-bearing, and we report all directions in Table 9.

Verified-leap behavior. Because C1 makes commit and rollback typed, replayable outcomes, we can read directly from the logs how often the leap fires and where the gate turns speculation away. The controller exercises the verified leap rather than defaulting to single-cycle steps. Over the full S-full run (the same run that serves as the typed arm of the ablation in Section 4.7), 1636 of the 1646 rounds reached a predictor transaction (the first ten ran at baseline before a committed history existed); these produced 1308 commits and 33 rollbacks (the remaining 295 not applicable, i.e. no admissible horizon $k \geq 2$ was available and the system took an ordinary single cycle), at a mean committed horizon of 2.47; the rollbacks are the cases where the four-level gate (Section 3.4) rejected a speculative outcome and restored the prior state. Two horizon quantities differ. The

requested horizon is the k the controller selects at S_t (mean 2.47 over commits); the *realized* saving of a committed leap is $k-1$, the number of ordinary guidance cycles skipped beyond the single cycle a normal step would already advance. Summed over the 1308 commits, the realized net saving is $\sum_t (k_t - 1) \approx 1924$ ordinary guidance cycles skipped through verified speculation (equivalently, the 1308 commits reach ≈ 3232 ordinary cycles in total, of which 1308 would have been taken by the baseline and 1924 are the net skip). We report these as descriptive counts from a single run, not as a controlled speedup claim: isolating wall-clock speedup would require a matched baseline with the leap disabled, which we leave to future work (Section 6). What is established here is narrower and verifiable from the logs: the gate fires, rolls back on failure, and commits multi-cycle horizons in the large majority of applicable transactions.

The rollbacks also show which level performs the filtering, which speaks to whether the gate is a genuine screen or a mere formality. Of the 33 rollbacks, none failed at L1 (bundle legality) or L2 (actual-output structural consistency): every rejected transaction produced a well-formed, cross-consistent speculative outcome and was turned away only by the semantic levels. L3 (blinded evaluator risk) rejected 6 and L4 (empirical development quality) rejected 27. The filtering therefore happens where it should, on measured evaluator risk and development-set quality of the real speculative outputs, not on format or routing. We do not overinterpret a 33-event sample, and we still lack the counterfactual that would show the rejected states would have harmed holdout quality had they committed (a leap-disabled control, left to future work); but the level breakdown is evidence that the gate discriminates on outcome quality rather than admitting everything that is merely well-typed.

Taken together, the benchmark, the ablation, and the leap logs converge on a single conclusion: the structured transport of C1 provides no predictive accuracy, yet everything the search does that a prose channel could not, namely verifiable leaping, leak-proof gating, and auditable rollback, rests on it. We turn next to what this implies for when the design is worth adopting.

5 Discussion

The core finding of this design is architectural: structuring the inter-agent channel is what makes a speculative leap safe and auditable. Because every committed transmission is a typed, causally addressed record, the controller can validate a multi-cycle outcome against the same schemas that govern ordinary execution, replay any feedback transition, and structurally prevent a forecast from reaching an evaluator. The verify-then-commit discipline means a wrong forecast is never a correctness risk, only a cost, since the transaction rolls back to an exact prior state and the system proceeds as the unaccelerated baseline would.

Our ablation (Section 4.7) sharpens what structured transport does and does not provide. It shows no measurable accuracy advantage: a free-text channel carrying the same information reaches the same hit rate in our single paired run, so we do not claim typing improves prediction. Its value is entirely in the correctness properties above: a state that can be checked against a schema, replayed deterministically, and structurally kept from leaking a forecast to an evaluator, all of which we establish by construction. This is a narrow and honest claim. A reader who only cares about predictive accuracy could use free text; a reader who needs the leap to be verifiable, auditable, and safe to roll back cannot, and that is the regime this paper targets.

The mechanism is not specific to alpha mining. Any dual-loop or multi-agent search whose serial cost is dominated by inter-agent feedback, and whose hand-offs can be given a typed schema, could in principle wrap its communication in the same three-layer transport and speculate over multi-cycle guidance. Two preconditions govern whether it *pays* rather than merely *applies*. The guidance

state must be low-dimensional and slowly varying enough to be predictable several cycles ahead; and, as in speculative decoding, verifying a multi-cycle speculative outcome must be cheaper than executing the cycles it replaces. The second is the binding one: where verification itself requires re-running the expensive machinery a cycle would (in our case a full backtest), a leap saves nothing, and our own setting does not yet demonstrate the economic condition empirically, consistent with our reporting only descriptive leap counts and no measured speedup. We therefore state transfer as a design conjecture rather than a demonstrated result: we evaluate exactly one system on one market, so transferability across systems, markets, or asset classes remains untested (Section 6). The alpha-discovery setting is a demanding instance because two coupled loops must stay cross-consistent and because a leaked forecast to an evaluator would compromise the independence of the quality gate; the visibility typing and separated skill libraries address exactly these risks.

A cost remains when speculation fails. If no horizon is admissible, or if every trial rolls back, the system pays the speculative and verification overhead for no committed progress. We therefore report this overhead directly rather than assuming a cost-free worst case, and we treat the quality constraint of Equation (2) as an empirically estimated bound with thresholds fixed before final testing.

When, then, is the transport worth adopting? Our own results answer bluntly, and not always in our favor. On the effectiveness axis the ablation shows typing provides no predictor accuracy or factor-quality advantage; the verified leap yields no measured wall-clock speedup (we report only descriptive counts, Section 4.7); and the returns are gross. For a team optimizing purely for returns or raw speed, these results do not justify the engineering cost of a typed transactional transport, and free text would serve equally well on the metrics the predictor optimizes. What the typed transport uniquely provides is a governable research pipeline: every inter-agent decision is a typed, causally-addressed, replayable record with commit/rollback provenance, so the exact trajectory that produced a deployed factor can be reconstructed and audited, and a speculative forecast is structurally barred from reaching a quality gate. This delivers decision *auditability* (the recorded trajectory can be replayed and inspected) rather than decision *reproducibility* in the strict sense: because the underlying agents are non-deterministic LLMs, replay reconstructs what happened from the logged tokens and states, it does not re-derive the decision from inputs. In a regulated or model-risk-governed quant setting, where decision auditability, audit trails, and separation of the forecasting policy from the evaluation standard are compliance requirements rather than conveniences, this correctness-by-construction is the payoff, and the null accuracy result is then the desirable outcome (auditability at no cost to effectiveness) rather than a disappointment. We regard this as the regime the design targets, rather than speed or returns. The correctness properties themselves are established by construction and argument; we have not yet stress-tested them empirically (Section 6), so this adoption case is a design argument pending the leakage and gate-efficacy experiments we identify as future work.

Two scope notes bound the portfolio-level result. First, the selected-portfolio returns carry substantial dispersion: on the same development-top-20 set whose median holdout Sharpe is +0.71 (Table 3), the *mean* holdout Sharpe is higher (+0.87) because a few portfolios reach much larger values, so the figure describes a positive but heavy-tailed distribution rather than a uniformly strong basket; we lead with the median precisely because it is the more conservative summary of the two. Second, absolute magnitudes are not comparable across studies on different universes and holdout windows; we make no claim of parity with results reported elsewhere and confine our comparison to the seven baselines run under our own shared backtest.

6 Limitations

Several limitations bound the scope of our claims, in three groups: the empirical scope of the evaluation, the calibration of the gate and its internal validation, and the status of the safety properties.

Empirical scope. We evaluate one universe (CSI 1000), so whether the learned trajectory structure transfers across markets or asset classes is untested; the transferability argument in Section 3.1.1 and the Discussion is made in principle rather than demonstrated empirically. All reported return and risk-adjusted figures are gross, before transaction costs, slippage, and turnover constraints: the cross-method comparison is internally consistent under the shared backtest, but the absolute magnitudes are not net-of-cost, and a cost- and turnover-adjusted evaluation is left to future work. Relatedly, the verified-leap counts of Section 4.7 are descriptive summaries of the committed run rather than a controlled speedup measurement; isolating a wall-clock speedup would require a matched leap-disabled baseline, which we also leave to future work. All results derive from a single end-to-end run under a fixed seed; we report them descriptively and do not estimate run-to-run variance, which would require repeated end-to-end runs and remains future work.

Gate calibration and internal validation. The gate thresholds $\tau_q, \tau_u, \tau_\alpha, \tau_m, \tau_g, \tau_p$ and the horizon set $\mathcal{K}$ are estimated on development data, so a miscalibrated gate could admit a low-quality state within the ϵ tolerance; rollback guarantees state safety, not quality optimality. When the trajectory is highly non-stationary, few horizons are admissible and the speculative overhead may not be recovered, and forecasting begins only once committed history exists, so early cycles run at baseline speed. Separating the predictor and evaluator skill libraries prevents forecast anchoring but does not by itself make each evaluator well-calibrated; that separation is a structural property, not a correctness proof. Two further caveats concern our own validation procedure rather than the method. The L4 gate scores every candidate leap on the same internal train-gate and leap-validation partitions across the 1341 transactions that reached a validation decision, which is an adaptive-validation (garden-of-forking-paths) hazard on that split; the reported holdout is free of it because the leap-validation split is disjoint from test and holdout and never enters a reported metric, but the mechanism may fit the internal split, which we do not correct for. And while the thresholds, splits, and designated primary endpoint are held fixed in `config_formal_v1.yaml` before the test and holdout splits are read, we do not deposit a timestamped or hashed pre-registration, so this freezing is a procedural claim about our workflow rather than an externally verifiable one.

Safety properties are established by construction. Visibility-typed leakage prevention, exact rollback, and predictor/evaluator library separation are analyzed structurally rather than demonstrated by a stress test such as an injected speculative-to-evaluator packet or a documented failure mode that shared libraries induce and separated ones prevent. The auditability the title foregrounds should be read in this light: it is a property of the typed, replayable record construction (the trajectory is logged and schema-checked by design), not an empirically stress-tested guarantee, and it means that a committed decision can be replayed and inspected rather than re-derived from inputs (Section 3.2 and the Discussion). We record these as acknowledged limitations rather than resolving them here.

7 Conclusion

We presented VST, a design that turns the communication layer of a dual-loop agentic alpha-discovery system into a substrate for verifiable multi-cycle acceleration. The enabling contribution is C1: a three-layer, A2A-compatible structured transport with deterministic unicast routing and visibility typing that preserves the existing payload paths and receiver arguments while making every transmission typed and replayable. This structure is the architectural prerequisite for the rest of the method; we position it as a correctness enabler rather than a source of predictive accuracy, consistent with our ablation. On the resulting committed trajectory, a single predictor with four typed heads forecasts the accumulated miner-directed guidance over a chosen horizon (C2); a four-level transactional gate validates the speculative outcome and commits it in a single logical step (absent process failure) or rolls it back to an exact prior state (C3); and separated skill libraries let the predictor and evaluators evolve without the predictor teaching the evaluators to agree with its forecasts (C4). *Structure comes before leaping*: the same typing that removes ambiguity from ordinary transmission is what makes future guidance learnable and a speculative leap verifiable. The experiments bear out the division of labor this structure implies: typing yields no accuracy, while the verified search it enables yields out-of-sample robustness. On a CSI 1000 out-of-sample holdout, the resulting system is the only one among eight methods (seven baselines and ours) to realize a positive median annualized return and the only one to retain a positive median backtest Sharpe at the factor level (bootstrap 95% CI entirely above zero); at the portfolio level, development-selected top-20 portfolios reach a median holdout Sharpe of 0.71 and a 13.3% median annualized return (gross of transaction costs, and selected on a test split inside the optimization horizon), with the smallest degradation from validation to holdout. This portfolio-level figure is a property of development-guided selection (the all-portfolio median holdout Sharpe is -0.23, and the development-to-holdout selection signal is a weak but positive Spearman $+0.19$), not of the raw portfolio population; taken together with the factor-level robustness, it indicates that the verified structured search generalizes rather than overfits within the scope of a single run.

References

[1] Y. Li, S. Liu, and B. Liu, "AI Trading's Alpha Singularity: Emergent Market Reasoning through Agent-to-Agent Self-Evolution," *arXiv preprint arXiv:2606.29194*, 2026.

[2] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "Metagpt: Meta programming for a multi-agent collaborative framework," *arXiv preprint arXiv:2308.00352*, 2023.

[3] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "Chatdev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, arXiv:2307.07924.

[4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen llm applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[5] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "Camel: Communicative agents for "mind" exploration of large language model society," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2303.17760.

[6] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," *arXiv preprint arXiv:2402.01680*, 2024.

[7] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning (ICML)*, 2023, arXiv:2211.17192.

[8] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," *arXiv preprint arXiv:2305.14325*, 2023.

[9] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian, Y. Qin, X. Cong, R. Xie, Z. Liu, M. Sun, and J. Zhou, "Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors," *arXiv preprint arXiv:2308.10848*, 2023.

[10] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," *arXiv preprint arXiv:2304.03442*, 2023.

[11] C. Qian, Z. Xie, Y. Wang, W. Liu, K. Zhu, H. Xia, Y. Dang, Z. Du, W. Chen, C. Yang, Z. Liu, and M. Sun, "Scaling large language model-based multi-agent collaboration," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2406.07155.

[12] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, 2024, arXiv:2308.11432.

[13] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou *et al.*, "The rise and potential of large language model based agents: A survey," *arXiv preprint arXiv:2309.07864*, 2023.

[14] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan, and H. D. Nguyen, "Multi-agent collaboration mechanisms: A survey of llms," *arXiv preprint arXiv:2501.06322*, 2025.

[15] T. Finin, R. Fritzson, D. McKay, and R. McEntire, "KQML as an agent communication language," in *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM)*, 1994, pp. 456–463.

[16] J. N. Foerster, Y. M. Assael, N. de Freitas, and S. Whiteson, "Learning to communicate with deep multi-agent reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016, arXiv:1605.06676.

[17] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, arXiv:2201.11903.

[18] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2305.10601.

[19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.03629.

[20] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2302.04761.

[21] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2303.11366.

[22] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *arXiv preprint arXiv:2303.17651*, 2023.

[23] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[24] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, arXiv:2203.02155.

[25] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "Deepseekmath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024.

[26] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *Nature*, vol. 645, pp. 633–638, 2025, arXiv:2501.12948.

[27] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.

[28] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, "Learning latent dynamics for planning from pixels," in *International Conference on Machine Learning (ICML)*, 2019, arXiv:1811.04551.

[29] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," in *International Conference on Learning Representations (ICLR)*, 2020, arXiv:1912.01603.

[30] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Nob Hill Publishing, 2017.

[31] S. Yu, H. Xue, X. Ao, F. Pan, J. He, D. Tu, and Q. He, "Generating synergistic formulaic alpha collections via reinforcement learning," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2023.

[32] B. K. Petersen, M. Landajuela, T. N. Mundhenk, C. P. Santiago, S. K. Kim, and J. T. Kim, "Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients," in *International Conference on Learning Representations (ICLR)*, 2021, arXiv:1912.04871.

[33] S. Yu, H. Xue, X. Ao, and Q. He, "A hybrid approach to formulaic alpha discovery with large language model assistance," *Frontiers of Computer Science*, vol. 20, no. 2, p. 2002316, 2026.

[34] Z. Tang, Z. Chen, J. Yang, J. Mai, Y. Zheng, K. Wang, J. Chen, and L. Lin, "Alphaagent: Llm-driven alpha mining with regularized exploration to counteract alpha decay," *arXiv preprint arXiv:2502.16789*, 2025.

[35] F. Feng, X. He, X. Wang, C. Luo, Y. Liu, and T.-S. Chua, "Temporal relational ranking for stock prediction," *ACM Transactions on Information Systems (TOIS)*, vol. 37, no. 2, pp. 1–30, 2019.

[36] X. Yang, W. Liu, D. Zhou, J. Bian, and T.-Y. Liu, "Qlib: An ai-oriented quantitative investment platform," *arXiv preprint arXiv:2009.11189*, 2020.

[37] Y. Li, S. Wang, H. Ding, and H. Chen, "Large language models in finance: A survey," in *Proceedings of the 4th ACM International Conference on AI in Finance (ICAIF)*, 2023, arXiv:2311.10723.

[38] S. Wu, O. Irsoy, S. Lu, V. Dabravolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, and G. Mann, "Bloomberggpt: A large language model for finance," *arXiv preprint arXiv:2303.17564*, 2023.

[39] H. Yang, X.-Y. Liu, and C. D. Wang, "Fingpt: Open-source financial large language models," *arXiv preprint arXiv:2306.06031*, 2023, finLLM Symposium at IJCAI.

[40] X. Yu, Z. Chen, Y. Ling, S. Dong, Z. Liu, and Y. Lu, "Temporal data meets llm – explainable financial time series forecasting," *arXiv preprint arXiv:2306.11025*, 2023.

[41] X.-Y. Liu, H. Yang, Q. Chen, R. Zhang, L. Yang, B. Xiao, and C. D. Wang, "Finrl: A deep reinforcement learning library for automated stock trading in quantitative finance," in *Deep Reinforcement Learning Workshop, NeurIPS*, 2020, arXiv:2011.09607.

[42] W. Zhang, L. Zhao, H. Xia, S. Sun, J. Sun, M. Qin, X. Li, Y. Zhao, Y. Zhao, X. Cai, L. Zheng, X. Wang, and B. An, "A multimodal foundation agent for financial trading: Tool-augmented, diversified, and generalist," *arXiv preprint arXiv:2402.18485*, 2024.

[43] Y. Xiao, E. Sun, D. Luo, and W. Wang, "Tradingagents: Multi-agents llm financial trading framework," *arXiv preprint arXiv:2412.20138*, 2024.

[44] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Transactions on Database Systems*, vol. 6, no. 2, pp. 213–226, 1981.

[45] M. Herlihy and J. E. B. Moss, "Transactional memory: Architectural support for lock-free data structures," in *Proceedings of the 20th Annual International Symposium on Computer Architecture (ISCA)*, 1993, pp. 289–300.

[46] G. Chevillon, "Direct multi-step estimation and forecasting," *Journal of Economic Surveys*, vol. 21, no. 4, pp. 746–785, 2007.

[47] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in AI safety," *arXiv preprint arXiv:1606.06565*, 2016.

[48] Z. Kakushadze, "101 formulaic alphas," *Wilmott*, vol. 2016, no. 84, pp. 72–81, 2016, arXiv:1601.00991.

Algorithm 1 AGGREGATE($\{G_{t+\ell}\}_{\ell=1}^{k}$) over one channel

initialize empty bundle B
for all field g in the channel schema **do**
  if g is set-valued **then**
    replay add/remove edits $g_{t+1}, \ldots, g_{t+k}$ in cycle order
    for each element, keep the most recent promote/demote, weighted by its support count
  else if g is numeric or bounded-categorical **then**
    $B[g] \leftarrow \mathrm{clip}(g_{t+k}, \mathrm{range}_{\mathcal{S}_B}(g))$
  else
    $B[g] \leftarrow g_{t+k}$ (latest committed advice text)
  end if
end for
return B

[49] Y. Shi, Y. Duan, and J. Li, "Navigating the alpha jungle: An LLM-powered MCTS framework for formulaic factor mining," *arXiv preprint arXiv:2505.11122*, 2025.

[50] S. Wang, H. Yuan, L. Zhou, L. M. Ni, H.-Y. Shum, and J. Guo, "Alpha-gpt: Human-ai interactive alpha mining for quantitative investment," in *Proceedings of EMNLP (System Demonstrations)*, 2025, arXiv:2308.00016.

[51] D. H. Bailey and M. López de Prado, "The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality," *The Journal of Portfolio Management*, vol. 40, no. 5, pp. 94–107, 2014.

[52] C. R. Harvey, Y. Liu, and H. Zhu, "... and the cross-section of expected returns," *The Review of Financial Studies*, vol. 29, no. 1, pp. 5–68, 2016.

[53] H. White, "A reality check for data snooping," *Econometrica*, vol. 68, no. 5, pp. 1097–1126, 2000.

A Aggregation Algorithm

The type-aware aggregator AGGREGATE of Equation (5) dispatches per field type, as summarized in Algorithm 1. Set-valued fields accumulate additions and removals in cycle order and collapse repeated promote/demote decisions on the same element to the most recent one, weighted by how many cycles supported it; numeric or bounded-categorical fields keep the latest value clipped to the schema's declared range; and string advice fields keep the most recent committed text. This makes the target bundle a deterministic function of the committed window, so the same prefix always yields the same target.

B Operational Runtime Side Effects

Beyond factor quality, structured transport moves a few operational quantities, which we report in full including the ones that do not favor it. Both arms ran on the same machine over the same period, which removes the most severe hardware confounds. Table 9 summarizes measurements derived from the closed-loop per-round records.

Table 9: Operational side effects of structured transport (same run logs, same period). The per-round wall-clock is observational (same machine and period, but not a controlled timing experiment); success-rate effects are small and mixed and are reported in both directions.

Quantity	S-off (free-text)	S-full (typed)
Per-round wall-clock, median (IQR)	385 s [363, 414]	**353** s [335, 373]
Portfolio structure-gate pass	0.985	**0.991**
Portfolio composite Sharpe, median	2.24	**2.28**
Single-factor candidate pass	**0.246**	0.128

Per-round wall-clock. The structured arm has a lower median per-round time in the logs (353 s versus 385 s, with overlapping interquartile ranges [335, 373] versus [363, 414]). We report the median, not the mean, because the free-text arm's mean is dominated by a heavy right tail of occasional multi-minute stalls that reflect transient API and scheduling variance rather than the transport. This figure is *observational*: the two arms shared a machine and period, but we did not run a controlled timing experiment (fixed inputs, repeated timed trials, isolated GPU load), so we cannot exclude API-latency or scheduling fluctuation as a contributor, and we present it as a suggestive side effect rather than a measured speedup claim.

Success rates, reported honestly in both directions. At the portfolio level the structured arm is marginally higher, both in the fraction of rounds whose composite passes the structure gate (0.991 versus 0.985) and in median composite Sharpe (2.28 versus 2.24). At the single-factor level, however, the free-text arm has the higher per-candidate evaluator pass rate (0.246 versus 0.128). We do not interpret the single-factor gap as evidence against structured transport, nor the portfolio gap as strong evidence for it: both differences are small, and the single-factor pass rate mixes how many candidates a run proposes with how strictly they are filtered. We present these as suggestive operational side effects, not as load-bearing claims of the paper.